Introduction

In 1976, renowned British statistician George E. P. Box wrote that *"all models are wrong, but some are useful"*, to comment on the incremental and interactive nature of scientific discovery (this exact statement appearing only three years later; Box, 1979). He further went on to say that

> *"In applying mathematics to subjects such as physics or statistics we make tentative assumptions about the real world which we know are false but which we believe may be useful nonetheless."*
>
> — George E. P. Box (1976)

This is the essence of what a *model* is: a convenient simplification of the intricacies observed in the real world (Cox, 1990). Even though these models may be merely statistical to explain what they observe, scientists using these models strive for a representation as faithful as possible, even going towards a *causal* understanding of the world to explain how it responds to active interactions (Pearl, 2009). But if all models are wrong, how can we find the ones that are useful? This question is of utmost importance since a bad model has the potential to be dangerous just as much as a good one may be useful (if not more). This is true now more than ever as artificial intelligence is progressing at such a fast pace and the safety of our models becomes critical (Bengio, 2023).

Finding a useful model is all the more difficult that it is typically based on *evidence*. In many cases, especially when the evidence is limited (but not only), multiple concurrent models may explain what we observe equally well and committing to a single candidate is prone to catastrophic failures beyond our observations (Hodges, 1987; Bengio, 2024). When faced with risky choices, a rational decision-maker tends to act as if it was maximizing some utility under *uncertainty* (Ma, 2022). To account for this epistemic uncertainty, it appears natural to take a *Bayesian approach*, which Box himself was a fervent advocate of (Box and Tiao, 1973), where the uncertainty is quantified by the *posterior distribution* over models given some observations. Adopting a Bayesian perspective to quantify the *structural uncertainty* of the models can be traced back to the research agenda proposed by Draper et al. (1987), and was addressed in part throughout the 1990s (Madigan and Raftery, 1994; Chatfield, 1995; Draper, 1995).

This now raises two questions: (1) how can we represent a statistical (or even causal) model of a system of interest, and (2) how can we represent a posterior distribution over those models? For the former, we choose the well-established framework of *Bayesian networks* (Pearl, 1988) for its flexibility and potential extensions to causality. For the latter though, defining a distribution over

the full characterization of a Bayesian network (*i.e.*, its parameters *and* its structure) proves to be particularly challenging. The main reason being that the structure of a Bayesian network is defined as a *directed acyclic graph*, thus the posterior must be a distribution over a vast number of discrete objects (directed graphs) with a complex acyclicity constraint. While we could leverage generic tools for Bayesian inference such as *Markov chain Monte Carlo* methods (Gelfand and Smith, 1990; Madigan and York, 1995), are there more appropriate approaches based on the combined strengths of modern generative models and deep learning (LeCun et al., 2015; Goodfellow et al., 2016)?

In this thesis, we introduce *generative flow networks* (GFlowNets; Bengio et al., 2021; Bengio et al., 2023) as a general framework to sample from distributions over discrete quantities that exhibit some form of *compositional* aspect, such as directed acyclic graphs. Generative flow networks are *discrete*, *sequential*, and *amortized* generative models in the following sense:

- *Discrete*: the objects generated by the GFlowNets are discrete, and will typically have a *compositional* structure as described in Chapter 2 (see Section 2.1.3 in particular for examples). This is an under-developed area in modern generative modeling, which has been primarily focused on continuous problems (Kingma and Welling, 2014; Rezende et al., 2014) or continuous relaxations of discrete ones (Maddison et al., 2017; Jang et al., 2017). We will see in Chapter 6 how GFlowNets can be extended beyond discrete state spaces, including to continuous cases;
- *Sequential*: GFlowNets treat generation as a *sequential decision making* problem, where an object is constructed one piece at a time. They are aligned with the recent trend of considering generative models as a sequential process (Song and Ermon, 2019; Brown et al., 2020; Lipman et al., 2023; Tong et al., 2024) as opposed to a static one (Goodfellow et al., 2014). In the case of GFlowNets, this is inspired by the framework of *reinforcement learning* (Sutton and Barto, 2018), and we will see in Chapter 5 that they are in fact closely related to entropy-regularized reinforcement learning (Haarnoja et al., 2017);
- *Amortized*: GFlowNets amortize the cost of sampling by training a policy that can eventually be used to generate new objects. This policy will typically be parametrized by a neural network, leveraging their generalization capabilities in different contexts with some shared structure, and are trained by minimizing some objectives we will introduce in Section 4.2.1. We will also see in Section 4.3 that GFlowNets are deeply rooted in the literature of *amortized variational inference* (Kingma and Welling, 2014).

Although GFlowNets are generative models in the "classical" sense of the term (under the generative/discriminative dichotomy in probabilistic modeling; Ng and Jordan, 2001), they are notably different from the more "modern" expectation of what generative models are: instead of being a model that learns a generative process from a dataset of examples (*e.g.*, image generation), GFlowNets rely on a predefined *reward function* that gives a notion of preference for some objects over others (*i.e.*, objects with a higher reward will be sampled more frequently than those with a lower reward). This is a setting frequently encountered in Bayesian inference, but also more generally in scientific discovery (Noé et al., 2019).

Outline

The first part of this thesis will be dedicated to the theory of generative flow networks in its generality. In Chapter 2 we introduce the problem of sampling from an *energy-based model*, which is a distribution defined up to an intractable normalization, that will serve as a through line during the first part. We will argue via multiple examples how sampling from a distribution over discrete and compositional objects can be seen as a sequential decision making problem, which in some cases falls into the framework of *maximum-entropy reinforcement learning* (MaxEnt RL).

In the general case though, this approach based on MaxEnt RL is biased and does not sample from the target distribution as expected. This motivates the introduction of GFlowNets as a general solution that does not suffer from the same bias. We first introduce in Chapter 3 some properties of *flow networks*, which serve as the foundations for GFlowNets, as well as various conditions to characterize them. Armed with this, we then present in Chapter 4 how to use flow networks in conjunction with appropriate *boundary conditions* for generative modeling. We show how the conservation laws characterizing the flow networks can be turned into practical learning objectives, and show novel convergence guarantees of the approximate distribution towards the target, making GFlowNets rooted in the field of *variational inference*.

Then in Chapter 5 we circle back and show that GFlowNets and MaxEnt RL are actually "two faces of the same coin", provided that the reward function is properly corrected. We show that many of the objectives introduced in Chapter 4 find natural counterparts in the MaxEnt RL literature. Finally in Chapter 6, through an excursion to *Markov chains*, we show how GFlowNets can be extended to general state spaces, including continuous ones, despite being originally created specifically for discrete settings.

The second part of this thesis will then focus specifically on the problem of *Bayesian structure learning*, that is to model the posterior distribution over Bayesian networks, using GFlowNets. In Chapter 7 we first present how GFlowNets can be used to sample from a distribution over directed acyclic graphs, by constructing a directed graph one edge at a time while efficiently guaranteeing acyclicity at every stage. This allows us to not only model the *marginal* posterior distribution over the structure of a Bayesian network alone, but also serves as the basis for modeling the *joint* posterior over the full characterization of a Bayesian network. In Chapter 8, we finally introduce two separate methods using GFlowNets to model the *joint* posterior distribution over the structure *and* parameters of a Bayesian network, including one with a single GFlowNet leveraging the extension to general state spaces introduced in Chapter 6.

Chapter 1

Background

In this chapter, we provide a brief overview of the necessary concepts of probabilistic modeling useful throughout this thesis. Other domains will also be introduced in further chapters, such as notions of graph theory in Section 2.2.2 and reinforcement learning in Section 2.3.1.

1.1. Bayesian networks

Probabilistic graphical models (Koller and Friedman, 2009; Murphy, 2023) offer a way to represent probability distributions of complex systems in a compact and modular fashion. Although there exists a variety of ways to encode modularity, such as factor graphs (Kschischang et al., 2001), we will focus primarily on *Bayesian network* in this thesis (Pearl, 1988), as they constitute the foundations for eventually representing causal knowledge, as we will see further in Section 1.5.

Before going further, we need to first recall the notion of *conditional independence*, which plays a critical role in Bayesian networks. Two random variables X and Y are said to be independent given a third random variable Z, and denoted by $X \perp\!\!\!\perp Y \mid Z$, if and only if

$$P(X, Y \mid Z) = P(X \mid Z)P(Y \mid Z). \tag{1.1.1}$$

This definition can be naturally extended to $Z \equiv \emptyset$, in which case X and Y are called *marginally independent*, and to cases where X, Y, or Z are sets of random variables. This characterization of conditional independence in (1.1.1) can be equivalently written as $P(X \mid Y, Z) = P(X \mid Z)$. In other words, if we observe the value of Z, then knowing something about Y does not provide any additional information about X.

1.1.1. Representation of conditional independencies

Abiding to the principle of modularity, a *Bayesian network* is a compact representation of a joint distribution between d random variables $X_1, \ldots, X_d$, describing some system of interest. Given a directed acyclic graph (DAG) G, where each node corresponds to one of those random variables,

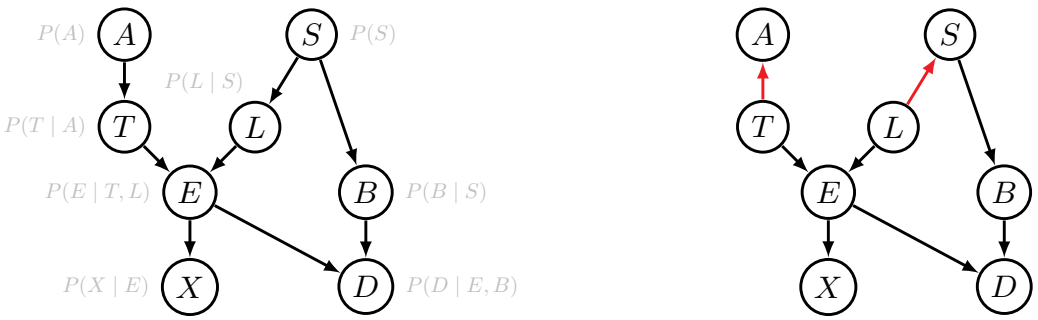


Figure 1.1. Factorization of the joint distribution $P(A, B, D, E, L, S, T, X)$ as a Bayesian network (Lauritzen and Spiegelhalter, 1988). (a) Illustration of the structure of a Bayesian network, with the corresponding conditional distributions. (b) An example of a Markov equivalent Bayesian network representing the same conditional independencies, where the edges $A \to T$ & $S \to L$ have been reversed.

the joint distribution factorizes according to the graph G in the following way

$$P_\theta(X_1, \ldots, X_d) = \prod_{i=1}^{d} P(X_i \mid \mathrm{Pa}_G(X_i); \theta_i), \tag{1.1.2}$$

where $\mathrm{Pa}_G(X_i)$ represents the parent random variables of X_i in G, and $\theta = (\theta_1, \ldots, \theta_d)$ are the parameters of the conditional distributions. The structure of the Bayesian network is typically fixed based on expert knowledge, although we will see in Section 1.4, and more generally in the second part of this thesis, that it is possible to learn it from data. Figure 1.1a shows an example of a Bayesian network.

Example 1.1.1 (Linear-Gaussian models)**.** One class of Bayesian networks we will study later in this thesis is called *linear-Gaussian models*. Given a DAG G, the conditional probabilities appearing in (1.1.2) are Normal distributions, whose mean is a linear combination of the parent variables. We can write such a model as a set of *structural equations* of the form

$$X_i := \sum_{j=1}^{d} \mathbb{1}(X_j \in \mathrm{Pa}_G(X_i))\theta_{ji} X_j + \varepsilon_i \qquad \text{where} \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma_i^2). \tag{1.1.3}$$

We use the convention "θ_{ji}" to align with the convention for adjacency matrices, effectively interpreting the parameters θ as a weighted adjacency matrix, with zero entries when there is no edge between two nodes. We can show that the joint distribution in that case is a multivariate Normal distribution with 0 mean, and whose covariance matrix depends on θ and σ_i^2's.

The factorization of the joint distribution $P_\theta(X_1, \ldots, X_d)$ according to the graph reveals a number of conditional independencies between sets of random variables. This is encoded in G via the notion of *d-separation* (Pearl, 1988), which is a purely graphical concept.

Definition 1.1.2 (d-separation)**.** An undirected path (*i.e.*, a sequence of edges in G, ignoring their orientations) is *d-separated* by a set of nodes X_C if it contains at least one of the following sub-structures:

(1) a *chain* of the form $A \to C \to B$, with $C \in X_C$;

(2) a *fork* of the form $A \leftarrow C \to B$, with $C \in X_C$;

(3) a *v-structure* of the form $A \to C \leftarrow B$, with neither C nor any of its descendents in X_C.

Two sets of nodes X_A and X_B are d-separated by X_C if any undirected path between nodes of X_A and X_B are d-separated by X_C.

Although this may seem like a highly complex criterion to verify, there exist algorithmic solutions to easily uncover d-separation (Shachter, 1998). When applying it to the structure of the Bayesian network, we can read conditional independencies (or lack thereof) in the joint distribution it represents. For example, one can verify that $A \not\perp\!\!\!\perp B \mid X$ in the joint distribution described by the Bayesian network in Figure 1.1a, even though $A \perp\!\!\!\perp B$. Being able to read conditional independencies from the graph is called the *global Markov property*.

Proposition 1.1.3 (Global Markov property)**.** *Let P be a distribution that factorizes according to a Bayesian network with structure G. Let X_A, X_B and X_C be three sets of random variables (nodes in G). If X_A and X_B are d-separated by X_C in G, then $X_A \perp\!\!\!\perp X_B \mid X_C$.*

Recall that this is a purely graphical (hence non-parametric) criterion: it only guarantees that if there exists d-separation, then there must be conditional independence in P_θ *regardless of the parameters* θ. However, there could be additional conditional independencies that are not reflected in the structure G but exist because of the specific parametrization θ. Finally, while it is tempting to interpret the direction of the edges of the graph as being "causal" influences of one variable on another, it is important to note that Bayesian networks only encode statistical relationships (conditional independencies), and not causal ones. Nevertheless, they serve as the foundations for causal models, which we will introduce in Section 1.5.

1.1.2. Markov equivalence

Since the DAG structure of a Bayesian network encodes the conditional independencies one can find in the joint distribution it represents, a question one could ask is whether this encoding is unique. The answer is unfortunately *no*: there exist multiple DAG structures that represent the same set of conditional independencies. Two DAGs G_1 and G_2 encoding the same set of conditional independencies are called *Markov equivalent*, and this equivalence relation creates a partition of the space of DAGs into Markov equivalence classes. Figure 1.1 depicts two DAGs that are Markov equivalent, despite having different structures. The following theorem gives a graphical criterion to check if two graphs are Markov equivalent.

Theorem 1.1.4 (Verma and Pearl, 1990)**.** *Two Bayesian networks with DAG structures G_1 and G_2 are Markov equivalent if and only if they have the same skeleton and the same v-structures.*

In this context, the skeleton of a directed graph corresponds to the same (undirected) graph with no orientation of its edges. Markov equivalence classes can be represented compactly as a *completed partially directed acyclic graph* (CPDAG), made of the skeleton and v-structures shared across all the members of the equivalence class. The existence of Markov equivalence reinforces our observation in the previous section that the directions of the arrows do not have a causal interpretation.

1.1.3. Sampling & probabilistic inference

Bayesian networks are *generative models*, in the sense that they model the joint distribution of an entire system. In particular, this means that if the structure and the parameters of the model are known, it is possible to generate virtual data from it. The acyclic nature of the graph suggests a very simple algorithm to sample from a Bayesian network called *ancestral sampling*, where the values of each random variable are sampled sequentially following a topological ordering of the DAG, based on the conditional probability distributions composing the factorization in (1.1.2).

But other than generating data, Bayesian networks are mainly used as a tool to answer questions about the system it is modeling. This is called *probabilistic inference*. Probabilistic inference refers very generally to the problem of estimating quantities of the form $\mathbb{E}_{P_\theta}[f(X_1, \dots, X_d)]$, where f is some function. In the context of Bayesian networks though, it will often refer to estimating conditional probabilities of the form $P_\theta(Z \mid X)$, where $\{X, Y, Z\}$ form a partition of $X_1, \dots, X_d$ (*i.e.*, we marginalize over the variables in Y). That's why inference is related to answering questions about a system: we want to know (probabilistically) something about an unknown quantity Z, given that we observed some other variables X, ignoring (marginalizing out) all other variables Y. The unobserved Z are called *latent variables*. Although it is possible to leverage the conditional independencies encoded by the Bayesian network to simplify $P_\theta(Z \mid X)$ in some special cases, in general computing this type of conditional distributions is NP-hard (Dagum and Luby, 1993). Therefore, we often resort to approximations such as *variational inference* as presented further.

1.1.4. Parameters learning

We have assumed thus far that the structure G of the Bayesian network and the parameters θ of its conditional distributions were known and fixed. However, it is possible to learn either of them from a dataset of observations $\mathcal{D} = \{\boldsymbol{x}^{(n)}\}_{n=1}^N$. While we will see in Section 1.4 how to learn G from data, a very common task is to learn the parameters θ of a Bayesian network with a known structure.

If we observe the whole system, meaning that each datapoint $\boldsymbol{x}^{(n)} \in \mathbb{R}^d$ in $\mathcal{D}$ is a complete assignment of all the random variables, then we can learn the parameters by maximizing the (log-)likelihood of the data available:

$$\widehat{\theta}_{\text{MLE}} = \arg\max_\theta \log P_\theta(\mathcal{D}) = \arg\max_\theta \sum_{n=1}^N \log P_\theta(\boldsymbol{x}^{(n)}). \tag{1.1.4}$$

Thanks to the decomposition of the joint distribution in a Bayesian network (1.1.2), this maximization can be carried out for each individual parameter θ_i in a completely independent way. However, if we only observe part of the system (*i.e.*, $\boldsymbol{x}^{(n)}$ is only a partial assignment, and some variables remain latent), then learning θ necessitates a step of probabilistic inference to "complete" the data. This is the idea behind the *(variational) expectation maximization* algorithm, which we will introduce in Section 1.2.3.

1.2. Variational inference

We consider a general inference problem of the form $P(\boldsymbol{z} \mid \boldsymbol{x})$, where $\boldsymbol{z}$ are latent variables and we observe $\boldsymbol{x}$. For example, this may be derived from a joint distribution $P(\boldsymbol{z}, \boldsymbol{x})$ described as a Bayesian network, with a known structure and parameterization. With some notable exceptions, for example when the Bayesian network has a near-tree structure (Zhang and Poole, 1996), computing this conditional distribution is typically intractable. While we could take a *sample-based* approach to inference if we had access to samples of $P(\boldsymbol{z} \mid \boldsymbol{x})$ (we will come back to it in Section 2.1.2), in this section we will focus on getting an approximation of this target distribution. We consider that the generative model $P(\boldsymbol{z}, \boldsymbol{x})$ is known and tractable (*i.e.*, if we had full information, then we could compute this joint probability).

1.2.1. Inference as optimization

Variational inference is the general principle of treating inference as an optimization problem. The goal is to approximate the intractable distribution $P(\boldsymbol{z} \mid \boldsymbol{x})$ with a tractable (and often simpler) distribution $Q_\phi(\boldsymbol{z})$ coming from a family of distributions $\mathcal{Q} = \{Q_\phi \mid \phi \in \Phi\}$ parametrized by ϕ. For example, the variational family $\mathcal{Q}$ may be the set of Normal distributions parametrized by their natural parameters $\phi = (\mu, \sigma^2)$. This approximation can be quantified with some notion of discrepancy D between the two distributions, which we seek to minimize

$$Q_{\phi^\star} = \underset{Q_\phi \in \mathcal{Q}}{\arg\min}\, D(Q_\phi(\boldsymbol{z}), P(\boldsymbol{z} \mid \boldsymbol{x})). \tag{1.2.1}$$

The distribution $Q_\phi(\boldsymbol{z})$ approximates the conditional $P(\boldsymbol{z} \mid \boldsymbol{x})$ *for a specific* $\boldsymbol{x}$, and therefore its dependency on $\boldsymbol{x}$ is implicit here; however, we will see in the next section how we can *amortize* inference and consider a distribution $Q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$ that depends explicitly on $\boldsymbol{x}$. To measure the difference between two distributions P and Q, there exists a broad class of divergences called *f-divergences* based on non-negative convex functions f such that $f(1) = 0$, defined by

$$D_f(P(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, Q(\boldsymbol{z})) = \int_{\boldsymbol{z}} Q(\boldsymbol{z}) f\left(\frac{P(\boldsymbol{z} \mid \boldsymbol{x})}{Q(\boldsymbol{z})}\right) d\boldsymbol{z}. \tag{1.2.2}$$

A popular example of divergence is the *Kullback-Leibler divergence*, also known as the KL divergence, which corresponds to the f-divergence associated with the convex function $f(x) = -\log x$:

$$\mathrm{KL}(Q(\boldsymbol{z}) \,\|\, P(\boldsymbol{z} \mid \boldsymbol{x})) = \int_{\boldsymbol{z}} Q(\boldsymbol{z}) \log \frac{Q(\boldsymbol{z})}{P(\boldsymbol{z} \mid \boldsymbol{x})} d\boldsymbol{z}. \tag{1.2.3}$$

A general property of f-divergences is that it achieves its minimum at 0 iff. both distributions are equal, justifying our choice to quantify the accuracy of the approximation. Minimizing the

discrepancy between $Q_\phi(\boldsymbol{z})$ and $P(\boldsymbol{z} \mid \boldsymbol{x})$ may seem daunting, since $P(\boldsymbol{z} \mid \boldsymbol{x})$ is intractable and yet necessary in the definition of a f-divergence like the KL divergence. Fortunately, we can rewrite the KL divergence to explicitly separate out the intractable evidence $P(\boldsymbol{x})$:

$$\mathrm{KL}(Q_\phi(\boldsymbol{z}) \,\|\, P(\boldsymbol{z} \mid \boldsymbol{x})) = \int_{\boldsymbol{z}} Q_\phi(\boldsymbol{z}) \log \frac{Q_\phi(\boldsymbol{z})}{P(\boldsymbol{z} \mid \boldsymbol{x})} = \underbrace{\int_{\boldsymbol{z}} Q_\phi(\boldsymbol{z}) \log \frac{Q_\phi(\boldsymbol{z})}{P(\boldsymbol{z}, \boldsymbol{x})}}_{\triangleq -\mathcal{J}(\phi)} + \log P(\boldsymbol{x}). \tag{1.2.4}$$

The quantity $\mathcal{J}(\phi)$ is called the *evidence lower-bound* (ELBO), and gets its name from the fact that it is a lower bound of the (log-)evidence $\log P(\boldsymbol{x}) \geq \mathcal{J}(\phi)$ (since the KL divergence is non-negative). It is clear that minimizing the KL divergence is exactly equivalent to maximizing the ELBO, which is more commonly written as

$$\mathcal{J}(\phi) = \mathbb{E}_{Q_\phi}\big[\log P(\boldsymbol{x} \mid \boldsymbol{z})\big] - \mathrm{KL}(Q_\phi(\boldsymbol{z}) \,\|\, P(\boldsymbol{z})). \tag{1.2.5}$$

To find the distribution $Q_\phi(\boldsymbol{z})$ that best approximates $P(\boldsymbol{z} \mid \boldsymbol{x})$ (in the sense of the KL divergence), it is therefore standard to obtain the variational parameters $\phi^\star$ that maximize $\mathcal{J}(\phi)$ using gradient-based methods.

1.2.2. Amortized inference

This variational approach considers that we perform inference for a single observation $\boldsymbol{x}$. If we wanted to do inference for a novel observation $\boldsymbol{x}'$, then we would have to go through the same exercise and find a completely separate variational distribution $Q'_\phi(\boldsymbol{z})$ that minimizes $\mathrm{KL}(Q'_\phi(\boldsymbol{z}) \,\|\, P(\boldsymbol{z} \mid \boldsymbol{x}'))$, and therefore solve a completely new optimization problem. This quickly becomes cumbersome, as each observation requires its own variational distribution.

Instead of treating each as being separate optimization problems, we could make use of some common information shared between observations to simplify inference. We can make the variational distribution $Q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$ depend explicitly on the observation $\boldsymbol{x}$, and learn a single set of variational parameters ϕ that would work for any observation $\boldsymbol{x}$. This process of sharing parameters across observations is called *amortization* (Gershman and Goodman, 2014; Amos, 2023). $Q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$ is known as an *inference model*, in contrast with the generative model $P(\boldsymbol{z}, \boldsymbol{x})$. For example, the inference model can be defined as a Normal distribution $Q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{z} \mid \mu_\phi(\boldsymbol{x}), \sigma^2_\phi(\boldsymbol{x}))$, where this time the parameters of the Normal distribution are functions of the observation $\boldsymbol{x}$ parametrized by ϕ.

1.2.3. Estimation of the parameters

So far, we have assumed that the generative model $P_\theta(\boldsymbol{z}, \boldsymbol{x})$ was known (*e.g.*, a Bayesian network with fixed structure and parametrization). In practice, we may want to also learn the parameters θ of this model along with the variational parameters of the inference model $Q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$, based on a dataset of observations $\mathcal{D} = \{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(N)}\}$. To estimate the parameters of a model in the presence of latent variables via maximum likelihood of $P_\theta(\mathcal{D})$, it is standard to use the *expectation maximization algorithm* (EM; Dempster et al., 1977). When this is coupled with a variational approximation of $P(\boldsymbol{z} \mid \boldsymbol{x})$, this is known as the *variational EM algorithm* (Neal and Hinton, 1998).

The ELBO in that case becomes

$$\mathcal{J}(\theta, \phi) = \sum_{n=1}^{N} \mathbb{E}_{Q_\phi(\boldsymbol{z}|\boldsymbol{x}^{(n)})}\big[\log P_\theta(\boldsymbol{x}^{(n)} \mid \boldsymbol{z})\big] - \mathrm{KL}(Q_\phi(\boldsymbol{z} \mid \boldsymbol{x}^{(n)}) \,\|\, P_\theta(\boldsymbol{z})) \tag{1.2.6}$$

where we made the dependency on both parameters θ of the generative model and ϕ of the inference model explicit. It is common practice to reweight $\mathcal{J}(\theta, \phi)$ by the number of observations in order to normalize the value of the objective.

Following closely the formulation of the EM algorithm, the ELBO may be maximized using coordinate ascend, alternating between (1) finding the variational parameters ϕ that lead to a faithful approximation of $P_\theta(\boldsymbol{z} \mid \boldsymbol{x})$ (*i.e.*, maximizing $\mathcal{J}(\theta, \phi)$ wrt. ϕ, E step), and (2) finding the parameters of the generative model θ that best explain the observations completed with $Q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$ (*i.e.*, maximizing $\mathcal{J}(\theta, \phi)$ wrt. θ, M step). However recently, powered by the advances in deep learning and automatic differentiation, it has been common practice to jointly maximize $\mathcal{J}(\theta, \phi)$ using gradient-based methods (Kingma and Welling, 2014; Rezende et al., 2014).

Estimation of the gradients of the ELBO. Optimization with gradient-based methods necessitates access to the gradients of $\mathcal{J}(\theta, \phi)$. In particular, $\nabla_\phi \mathcal{J}(\theta, \phi)$ requires computing gradients of quantities of the form $\mathbb{E}_{Q_\phi}[f(\boldsymbol{z})]$, with appropriate functions f (*e.g.*, $f(\boldsymbol{z}) = \log P_\theta(\boldsymbol{x}^{(n)} \mid \boldsymbol{z})$ in the first term of (1.2.6)). In order to account for the dependency of the distribution Q_ϕ, over which the expectation is taken, on the parameters ϕ when computing the gradient $\nabla_\phi \mathbb{E}_{Q_\phi}[f(\boldsymbol{z})]$, we can push the parameters ϕ inside the expectation via a change of variables (Rubinstein, 1992; Schulman et al., 2015). To make things more concrete, suppose that the variational family is of the form $Q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{z} \mid \mu_\phi(\boldsymbol{x}), \sigma^2_\phi(\boldsymbol{x}))$. We can rewrite an expectation wrt. Q_ϕ as

$$\mathbb{E}_{Q_\phi}[f(\boldsymbol{z})] = \mathbb{E}_{\varepsilon \sim \mathcal{N}(0,1)}[f(\mu_\phi(\boldsymbol{x}) + \sigma_\phi(\boldsymbol{x})\varepsilon)]. \tag{1.2.7}$$

With this, we can then compute the gradient of this expectation wrt. ϕ by simply differentiating inside the expectation as the distribution $\mathcal{N}(0, 1)$ no longer depends on ϕ

$$\nabla_\phi \mathbb{E}_{Q_\phi}[f(\boldsymbol{z})] = \mathbb{E}_{\varepsilon \sim \mathcal{N}(0,1)}[\nabla_\phi f(\mu_\phi(\boldsymbol{x}) + \sigma_\phi(\boldsymbol{x})\varepsilon)]. \tag{1.2.8}$$

This is known as the *pathwise derivative* (Mohamed et al., 2020), or the *reparametrization trick* (Kingma and Welling, 2014). This leads to a natural estimation of the gradient of the ELBO, based on a Monte-Carlo estimate of (1.2.8). Estimating the gradient $\nabla_\theta \mathcal{J}(\theta, \phi)$ on the other hand is typically easier since there is no need for reparametrization.

This is an effective way to estimate the gradients of the ELBO when the latent variables are continuous. But when $\boldsymbol{z}$ is discrete, variational inference becomes more challenging as there is in general no direct counterpart of the reparametrization trick in the discrete case. If the number of possible values the latent variables $\boldsymbol{z}$ may take is small enough, the expectations appearing in (1.2.6) can be computed analytically. However, enumeration of the values taken by $\boldsymbol{z}$ is rarely feasible, and the presence of discrete latent variables often necessitates continuous relaxations of the problem (Jang et al., 2017; Maddison et al., 2017), the use of score-function estimators that generally suffer from high variance (Williams, 1992; Kleijnen and Rubinstein, 1996; Rubinstein et al., 1996), or biased estimations of the gradients via the straight-through estimator (Bengio et al., 2013).

1.2.4. Wake-sleep algorithm

The difficulty of maximizing the ELBO in cases where the latent variables are discrete comes from the estimation of $\nabla_\phi \mathcal{J}(\theta, \phi)$ (for the E step in a variational EM setting). This is due to the fact that the expectations are taken wrt. the inference model Q_ϕ in the KL divergence. The KL divergence being asymmetric, choosing to minimize $\mathrm{KL}(Q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, P_\theta(\boldsymbol{z} \mid \boldsymbol{x}))$ in this order (also called the *reverse KL*) may seem arbitrary since any measure quantifying the difference between these two distribution would do. While this is considered standard in variational inference, we could use any other f-divergence with another convex function f.

Similar to variational EM, the *wake-sleep algorithm* (Hinton et al., 1995; Bornschein and Bengio, 2015) minimizes the difference between the conditional distribution induced by the generative model $P_\theta(\boldsymbol{z}, \boldsymbol{x})$ and the inference model $Q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$ by alternating between two phases:

(1) A *wake phase*, where the reverse KL divergence $\mathrm{KL}(Q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, P_\theta(\boldsymbol{z} \mid \boldsymbol{x}))$ is being minimized wrt. the parameters of the generative model θ. This is exactly equivalent to the M step in variational EM (*i.e.*, maximizing the ELBO wrt. θ);

(2) A *sleep phase*, where this time the *forward KL* divergence is being minimized (as opposed to the reverse KL in the E step of variational EM) wrt. the parameters of the inference model ϕ. The forward KL divergence is the f-divergence with $f(x) = x \log x$, corresponding to $\mathrm{KL}(P_\theta(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, Q_\phi(\boldsymbol{z} \mid \boldsymbol{x}))$. Following the same argument as in Section 1.2.1, this corresponds to maximizing the log-likleihood objective

$$\mathcal{J}_{\text{sleep}}(\phi) = \mathbb{E}_{P_\theta(\boldsymbol{z}, \boldsymbol{x})}\big[\log Q_\phi(\boldsymbol{z} \mid \boldsymbol{x})\big]. \tag{1.2.9}$$

Unlike using the reverse KL divergence for the update of both θ and ϕ in variational EM, this does not require the inference model to be reparametrizable anymore, and therefore is readily applicable to cases where the latent variables are discrete. However, unlike the EM algorithm, wake-sleep has no guarantee to converge to a maximum of the likelihood (Neal and Dayan, 1997).

1.3. Bayesian statistics

When we introduced Bayesian networks, we assumed that they were parametrized by some θ that one could maybe learn from data. These learned parameters may sometimes be unreliable, especially when the amount of data is relatively small and the risk of overfitting to that dataset is high. In those cases, it is beneficial to take a *Bayesian approach* and to treat the parameters as random quantities themselves. The problem of Bayesian inference is therefore to characterize the *posterior* distribution $P(\theta \mid \mathcal{D})$, defined by *Bayes' rule* as

$$P(\theta \mid \mathcal{D}) = \frac{P(\mathcal{D} \mid \theta) P(\theta)}{P(\mathcal{D})}, \tag{1.3.1}$$

where $P(\theta)$ is called a *prior* over the parameters θ, $P(\mathcal{D} \mid \theta)$ the *likelihood* of the dataset, and $P(\mathcal{D})$ the *marginal likelihood* (or evidence), which is obtained by marginalizing over all possible θ

$$P(\mathcal{D}) = \int_\theta P(\mathcal{D} \mid \theta) P(\theta) d\theta. \tag{1.3.2}$$

In general, computing the marginal likelihood above is intractable, making the computation of the posterior distribution itself intractable as well. A notable (but rare) exception is when the likelihood and the prior distribution are "conjugate" of one another, in which case the posterior $P(\theta \mid \mathcal{D})$ remains in the same family of distributions as the prior. While we will briefly mention a variational approach to approximate the posterior in Section 1.3.2, approximating distributions known up to a normalization constant, as is the case for $P(\theta \mid \mathcal{D})$, will be central in the first part of this thesis.

1.3.1. Bayesian model averaging

More generally than modeling uncertainty over the parameters of the same model, the Bayesian approach applies to complete hypotheses h as well, where the posterior distribution $p(h \mid \mathcal{D})$ is over models of data. For example, a hypothesis might be the full specification of the parameters along with the structure of a Bayesian network; this will be studied extensively in the second part of this thesis. We will assume that hypotheses $h \in \mathcal{H}$ belong to a space which may be discrete (*e.g.*, the DAG G of a Bayesian network), continuous (*e.g.*, the parameters θ of a Bayesian network with fixed structure), or a mixture of both (*e.g.*, both the structure G and the parameters θ are unknown).

Bayesian model selection. Suppose that we want to choose between two hypotheses h_1 and h_2 which one best fits the data available. If we know the posterior distribution, one way we can do that is to choose the hypothesis with the highest posterior probability. Although we saw that this distribution is typically intractable, we can fortunately compare the ratio of probabilities and cancel out the intractable marginal likelihood

$$\frac{P(h_1 \mid \mathcal{D})}{P(h_2 \mid \mathcal{D})} = \frac{P(\mathcal{D} \mid h_1)}{P(\mathcal{D} \mid h_2)} \frac{P(h_1)}{P(h_2)}. \tag{1.3.3}$$

The ratio of likelihood terms is also called the *Bayes factor* (Kass and Raftery, 1995), and serves as a measure to quantify the strength of the evidence for either hypotheses in case of a uniform prior (Jeffreys, 1948). More generally, we can look over the whole family of hypotheses and pick the one with highest posterior (log-)probability

$$\widehat{h} = \arg\max_{h \in \mathcal{H}} \log P(h \mid \mathcal{D}) = \arg\max_{h \in \mathcal{H}} \log P(\mathcal{D} \mid h) + \log P(h). \tag{1.3.4}$$

When the hypothesis $h \equiv G$ is the structure of a Bayesian network, and $P(\mathcal{D} \mid G)$ then corresponds to the marginal likelihood (where the parameters θ have been marginalized out), this is also called *empirical Bayes* (Carlin and Louis, 2000).

Bayesian model averaging. Instead of selecting a single hypothesis with maximum posterior probability though, we could average out predictions based on *all* the hypotheses, weighted by the posterior. We can do this by computing the *posterior predictive* distribution of a new datapoint x_{new}

$$P(x_{\text{new}} \mid \mathcal{D}) = \int_h P(x_{\text{new}} \mid h) P(h \mid \mathcal{D}) dh, \tag{1.3.5}$$

provided that $x_{\text{new}} \perp\!\!\!\perp \mathcal{D} \mid h$; we will come back to this condition (and importantly, to failure cases) in Section 7.6.3. This holds for example when the hypothesis $h \equiv \theta$ are the parameters of a Bayesian network with fixed structure.

Bayesian model combination. While averaging the predictions from multiple models is reminiscent of ensembling in frequentist statistics (Breiman, 1996), the Bayesian approach remains fundamentally different. Bayesian model averaging relies on the implicit assumption that the data was generated from exactly one "true" model (Reichelt et al., 2024; Bernardo and Smith, 2009). The soft weights $P(h \mid \mathcal{D})$ only reflect a statistical inability to distinguish it based on limited data (Minka, 2000). Therefore, in the limit of large *independent and identically distributed (iid.)* data, the posterior will typically concentrate around a single model *if this true model is in the family* $\mathcal{H}$ (Clydec and Iversen, 2013), effectively ignoring incoherent hypotheses and reducing (1.3.5) to the prediction of a single model.

But most importantly, albeit theoretically grounded, predictions made with Bayesian model averaging tend to underperform when compared to frequentist ensembles (Domingos, 2000). It has been conjectured that this is a consequence of the hypothesis class not being expressive enough to capture all the subtleties of the real world; the "true" model, assuming it exists, is not in $\mathcal{H}$. To address this issue while still adhering to the Bayesian principle, multiple models may be *combined* together (Monteith et al., 2011; Kim and Ghahramani, 2012).

1.3.2. Variational Bayes

The variational approach presented in Section 1.2.3 allows us to find the best parameters θ of the generative model with some latent variables $\boldsymbol{z}$. What if the latent variables of interest are the model parameters θ themselves though, as it is the case in Bayesian inference? For example if the parameters can be naturally broken down into $\theta = (\theta_1, \theta_2)$, we can choose a variational family of the form $Q_\phi(\theta) = Q_{\phi_1}(\theta_1)Q_{\phi_2}(\theta_2)$ (mean-field approximation), ignoring the dependencies between θ_1 & θ_2 in the posterior approximation.

Using a similar argument as in Section 1.2, it can be shown that the KL divergence between the approximation $Q_\phi(\theta)$ and the (intractable) posterior $P(\theta \mid \mathcal{D})$ can be decomposed as

$$\mathrm{KL}(Q_\phi(\theta) \,\|\, P(\theta \mid \mathcal{D})) = -\mathcal{J}(\phi_1, \phi_2) + \log P(\mathcal{D}), \tag{1.3.6}$$

where $\mathcal{J}(\phi_1, \phi_2)$ is an evidence lower-bound (ELBO) that can be written as

$$\mathcal{J}(\phi_1, \phi_2) = -\mathrm{KL}(Q_{\phi_1}(\theta_1) \,\|\, \widetilde{Q}_1(\theta_1)) + C_2(\phi_2) \tag{1.3.7}$$

$$\text{where} \quad \widetilde{Q}_1(\theta_1) \propto \exp\left(\int_{\theta_2} Q_{\phi_2}(\theta_2) \log P(\mathcal{D} \mid \theta_1, \theta_2) d\theta_2\right), \tag{1.3.8}$$

and $C_2(\phi_2)$ is only a function of ϕ_2 (independent of ϕ_1). In a completely symmetric fashion, it is also easy to show that the ELBO can be written as $\mathcal{J}(\phi_1, \phi_2) = -\mathrm{KL}(Q_{\phi_2}(\theta_2) \,\|\, \widetilde{Q}_2(\theta_2)) + C_1(\phi_1)$, where $\widetilde{Q}_2(\theta_2)$ is defined similarly to (1.3.8), but replacing Q_{ϕ_2} by Q_{ϕ_1}, and integrating wrt. θ_1.

We can find the best approximation of the posterior $P(\theta \mid \mathcal{D})$ (in the sense of the KL divergence) by maximizing the ELBO $\mathcal{J}(\phi_1, \phi_2)$ in (1.3.6). Loosely inspired by variational EM in Section 1.2.3, we can maximize it using coordinate ascent, by repeating (1) minimizing $\mathrm{KL}(Q_{\phi_1}(\theta_1) \,\|\, \widetilde{Q}_1(\theta_1))$ wrt. ϕ_1 given a fixed Q_{ϕ_2}, and (2) minimizing $\mathrm{KL}(Q_{\phi_2}(\theta_2) \,\|\, \widetilde{Q}_2(\theta_2))$ wrt. ϕ_2 given a fixed Q_{ϕ_1}. This

algorithm is called *Variational Bayes* (Beal, 2003). In some cases, one of the minimization steps above can be performed analytically.

1.4. Structure learning

We saw in Sections 1.1.4 & 1.2.3 how to learn the parameters θ based on a dataset of (possibly partial) observations $\mathcal{D}$, provided that the structure G of the Bayesian network is known and fixed. In this section, we will go one step further and consider the "inverse problem" of learning the structure G of the Bayesian network from a dataset $\mathcal{D}$. This is an important problem in scientific discovery, since uncovering (potentially novel) statistical dependencies in a system is the main objective, and is at least as important as using the model itself for inference.

1.4.1. Faithfulness & identifiability

To learn the structure of a Bayesian network, it is necessary to have a guarantee that any conditional independencies that can be found from data are going to be reflected in the graph that is eventually found. Otherwise, this inverse problem is ill-posed no matter how much data is available to accurately detect conditional independence. This is called the "faithfulness" of the data-generating distribution.

Definition 1.4.1 (Faithfulness)**.** A distribution P is *faithful* to a DAG G if any conditional independence in p is reflected in the d-separations in G. Let A, B and C be three sets of nodes in G, with their respective sets of random variables X_A, X_B and X_C. If $X_A \perp\!\!\!\perp X_B \mid X_C$, then A and B are d-separated by C in G.

Faithfulness can be seen informally as the "reverse" of the global Markov property in Proposition 1.1.3. But even though the global Markov property is satisfied for any distribution represented by a Bayesian network, the notion of faithfulness is not automatic.

Example 1.4.2 (Violation of faithfulness)**.** Consider the joint distribution $P(X, Y, Z)$ described by the Bayesian network in Figure 1.2a. The structure of the Bayesian network follows the "natural" description of the conditional distributions, with Y depending on X, and Z depending on both X & Y. The graph G being complete, it does not encode any (non-trivial) conditional independencies a priori. However, one can show that the joint distribution $P(X, Y, Z)$ is a multivariate Normal distribution, whose only conditional independencies are $X \perp\!\!\!\perp Z$ and $X \not\perp\!\!\!\perp Z \mid Y$. Therefore, the distribution p is not faithful to G, and any attempt to learn the structure of the Bayesian network from P would result at best in the DAG in Figure 1.2b.

Thankfully, faithfulness is guaranteed for *almost* all distributions P that factorize according to some DAG, in the sense that the set of distributions for which faithfulness is violated is of measure zero (Koller and Friedman, 2009, Theorem 3.5); in the example above, the conditional distributions were carefully crafted to be unfaithful (Spirtes et al., 2000, Theorem 3.2). Therefore throughout this thesis, we will always assume faithfulness without explicitly mentioning it again.

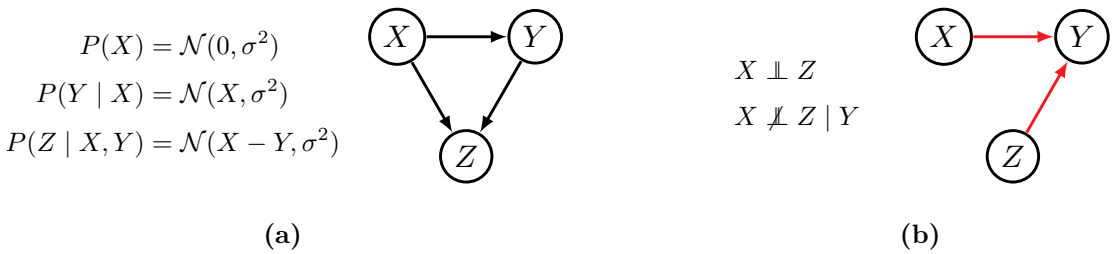


Figure 1.2. Example of a distribution violating the faithfulness assumption. (a) Example of a Bayesian network with its conditional distributions in the decomposition of $P(X, Y, Z)$. (b) The only conditional independence statements one can extract from $P(X, Y, Z)$ are $X \perp\!\!\!\perp Z$ & $X \not\perp\!\!\!\perp Z \mid Y$. In a fully non-parametric way, this DAG is the only one representing exactly both statements.

Going back to the problem of finding the structure of a Bayesian network, the following theorem shows that if we had complete access to the distribution P, then we could recover the DAG *up to Markov equivalence*, under faithfulness.

Theorem 1.4.3 (Structure identifiability)**.** *If a distribution P is faithful to $G^\star$, then the Markov equivalence class of $G^\star$ is identifiable from P.*

Shy of any assumption, the caveat of recovering the DAG only up to Markov equivalence has to be expected, since Markov equivalent structures encode the same conditional independencies. With additional assumptions on the data generating process, stronger forms of identifiability are possible, sometimes even of a single DAG the distribution is faithful to. For example, if P is a non-linear additive noise model, where the structural equation model for each variable can be written as

$$X_i := f_i(\mathrm{Pa}_{G^\star}(X_i)) + \varepsilon_i \qquad \text{with} \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma_i^2), \tag{1.4.1}$$

and where the functions f_i are non-linear and depend on the parent variables of X_i, then the DAG $G^\star$ is identifiable from P (Peters et al., 2014). Another classic example of identifiability of a single DAG is in the case where P is a linear-Gaussian model with equal variance (*i.e.*, $\sigma_i^2 = \sigma^2$ in (1.1.3); Peters and Bühlmann, 2014). We can also further identify the structure more precisely if we have access to experimental data (Hauser and Bühlmann, 2012); see Section 1.5.1 for an informal example.

1.4.2. Score-based methods

If we knew P completely, then this would lead to a simple strategy for structure learning (again, up to Markov equivalence): find the conditional independencies encoded in P, and construct a structure that follows them (in the sense of d-separation). However in practice, we never have access to P itself, but only to a finite dataset of observations $\mathcal{D}$. *Constraint-based* methods approach structure learning in a similar way, by testing conditional independencies from data using frequentist independence tests (Spirtes et al., 2000; Pearl, 2009).

Of greater interest in this thesis, *score-based* methods on the other hand treat the problem of learning the structure of the Bayesian network as an optimization problem. For some score function "score$(G; \mathcal{D})$" that quantifies how well G fits the dataset, the problem becomes to find a DAG that

maximizes this score:

$$\widehat{G} = \operatorname*{arg\,max}_{G \in \mathrm{DAG}} \mathrm{score}(G; \mathcal{D}). \tag{1.4.2}$$

Score functions. The most natural example of score function is the *likelihood score*, which corresponds to the log-likelihood of the maximum likelihood model

$$\mathrm{score}_L(G; \mathcal{D}) \triangleq \max_{\theta} \log P(\mathcal{D} \mid G, \theta) = \log P(\mathcal{D} \mid G, \widehat{\theta}_{\mathrm{MLE}}). \tag{1.4.3}$$

This score has the notable drawback of inevitably favoring dense structures (similar effect as standard overfitting in machine learning), meaning that using this score always leads to a trivial structure where all the nodes are connected together (Koller and Friedman, 2009). To mitigate this issue, it is possible to regularize this score by the number of free parameters in G, in order to penalize dense structures. The *Bayesian information criterion (BIC) score* (Schwarz, 1978) is one example. Another example of regularized score that we will use extensively in Chapter 7 is the so-called *Bayesian score*, which treats structure learning as a Bayesian model selection problem (introduced in Section 1.3.1) over the entire space of DAGs, with the score being

$$\mathrm{score}_B(G; \mathcal{D}) \triangleq \log P(\mathcal{D} \mid G) + \log P(G). \tag{1.4.4}$$

Although computing the marginal likelihood $P(\mathcal{D} \mid G)$ is in general intractable, we will see in Appendix B.1.1 that this score can be computed analytically for some classes of Bayesian networks (*e.g.*, linear-Gaussian models, or discrete Bayesian networks with Dirichlet prior).

Optimization of the score. Albeit conceptually simple, the optimization problem in (1.4.2) is extremely challenging in practice for two reasons:

(1) The space of DAGs is discrete, meaning that we cannot use any of the conventional tools from continuous optimization, such as gradient-based methods;

(2) The space of DAGs is combinatorially large. The number of DAGs over d nodes one can build is of the order $2^{\Theta(d^2)}$ (Robinson, 1973; Rodionov, 1992). To give some perspective, there are about 10^{18} DAGs over $d = 10$ nodes, and about 10^{72} DAGs over $d = 20$ nodes.

This means that this combinatorial optimization problem is NP-hard. Worse, even if we restrict the search space to only DAGs having at most p parents (with $p \geq 2$), then this problem remains NP-hard (Chickering, 1996). Therefore it is necessary to fall back on heuristic approaches in order to find a DAG maximizing the score (Cooper and Herskovits, 1992), such as evolutionary algorithms (Wong and Leung, 2004), or greedy approaches (Chickering, 2002; Tsamardinos et al., 2006).

1.4.3. Continuous relaxations

Another approach that has gained popularity recently is to relax the combinatorial optimization problem (1.4.2) into a continuous and constrained optimization problem. This stems from the observation of Zheng et al. (2018) that a graph G with adjacency matrix A is a DAG if and only if

$$\mathrm{Tr}\left(\exp(A)\right) = d, \tag{1.4.5}$$

where d is the number of nodes in G, and $\exp(A)$ is the matrix exponential. The function $\operatorname{Tr}\left(\exp(A)\right)$ effectively counts the number of cycles in the graph, reweighted by their length, and it being equal to d means that the only "cycles" that exist in a DAG are those of length 0. Other characterizations of DAGs based on operations on the adjacency matrix also exist (Yu et al., 2019; Yu et al., 2021; Bello et al., 2022). If the score function can be expressed as a function of the adjacency matrix, which is the case for the scores mentioned in the previous section, we can treat (1.4.5) as a constraint, and search over the space of all relaxed adjacency matrices

$$\max_{A \in [0,1]^{d \times d}} \operatorname{score}(A; \mathcal{D}) \tag{1.4.6}$$

$$\text{s.t. } \operatorname{Tr}\left(\exp(A)\right) = d. \tag{1.4.7}$$

This constrained optimization problem can then be solved, this time with gradient-based methods, provided that the score is differentiable wrt. A, for example using the *augmented Lagrangian* algorithm (Bertsekas, 2016). While Zheng et al. (2018) applied this exclusively to the discovery of linear-Gaussian models, we extended this idea to non-linear models (Lachapelle et al., 2020).

1.4.4. Identifiability & Bayesian perspective

The result of Theorem 1.4.3 only guarantees the identifiability of the Markov equivalence class of $G^\star$ if we know P itself. However in practice in structure learning, we don't have access to the whole distribution, but only to a finite dataset $\mathcal{D}$ of samples from that distribution; identifiability really is an asymptotic notion, in the limit of infinite data. The risk in practice is two-fold: (1) we may not have enough data in $\mathcal{D}$ to fully identify the Markov equivalence class (MEC) of $G^\star$, and (2) even if we identify it, a structure learning algorithm may choose an arbitrary element of the equivalence class, since there is non-identifiability *within* the MEC.

To get a more comprehensive view, we could instead take a Bayesian approach as we described in Section 1.3, and get a "soft" notion of identification via the posterior distribution (Cole, 2020). In cases where the model is not identifiable, this gets reflected in the posterior distribution $P(G \mid \mathcal{D})$, which then has support over the whole class of non-identifiability (*e.g.*, the MEC in the limit of infinite data), provided that the prior puts non-zero probability on these models. But the prior can provide additional information to break symmetry in case of non-identifiability (Lindley, 1972); *e.g.*, it has become increasingly popular to consider sparsity inducing penalties in order to prove identifiability (Lachapelle et al., 2024). If the model is identifiable (*e.g.*, to a single DAG, see examples in Section 1.4.1), the posterior distribution then is consistent and converges to a Dirac distribution, under some regularity conditions (*e.g.*, via the Bernstein-von Mises theorem; Ma, 2022).

And this is without counting on the main advantage of Bayesian inference: to account for uncertainty over models, especially in cases where the dataset is finite. All these advantages motivate our Bayesian approach for structure learning, which we will detail in the second part of this thesis. However, this will come at the cost of characterizing an intractable posterior distribution, which we will have to approximate.

1.5. Causality

In Section 1.1, we emphasized that the directed edges of a Bayesian network do not encode any notion of direct causal relationship. These models were developed only to describe statistical relationships of a system that we *passively* observe. A *causal model*, on the other hand, extends the notion of Bayesian network where this time the edges of the DAG correspond to direct causal relationships. Although functionally similar, a causal model allows us to not only passively observe a system, but also reason about *actively* experimenting on it, just like how a scientist would in order to verify a hypothesis. This is done via the notion of *interventions*. Causal models also allow us to reason about fictional situations called "counterfactuals" (Pearl, 2009), which we will not detail.

1.5.1. Interventions

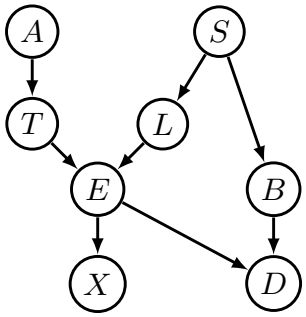
Let's start with an illustrative example to highlight the fundamental differences between observing and acting on a system. Suppose that we have two random variables of interest: a variable X encoding the weather (*e.g.*, as a categorical variable), and Y encoding the measurement of a barometer (continuous measure of the atmospheric pressure). From historical data, passively observing the weather and recording the corresponding value from the barometer every day, it is clear that there exists a dependence between these two variables (high pressure correlates with good weather), and in particular that only the DAGs $X \to Y$ or $Y \to X$ may encode the system. However based on our discussion on Markov equivalence in Section 1.1.2, these two DAGs encode the same conditional independencies (by Theorem 1.1.4), and therefore may equally model the system.

If all we could do was passively observe X & Y, then this is the only conclusion we can draw (Theorem 1.4.3). But suppose that we actively change the measurement of the barometer (*e.g.*, by moving the needle towards higher values). This (unfortunately) has no effect on the weather. The operation of actively changing the value of Y is called performing an *intervention* on Y, and it induces a distribution on (X, Y) that has the property of leaving the marginal over X unchanged (*i.e.*, intervening on Y does not affect our observational distribution over X). Therefore we can confidently conclude that the causal model that best describes the system is $X \to Y$ (*i.e.*, the weather *causes* the measurement on the barometer).

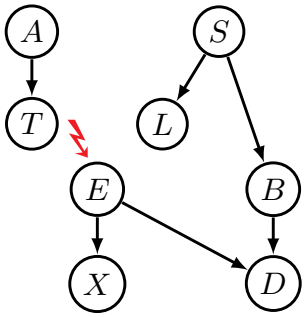
We saw that intervening on the system induces a new joint distribution over the variables of interest. We will now formalize this idea more generally for a causal model with a DAG structure G. For a set of indices $\mathcal{I}$, if the pair $\{X_{\mathcal{I}}, X_{-\mathcal{I}}\}$ forms a partition of $X_1, \ldots, X_d$, then the interventional distribution of acting on the variables $X_{\mathcal{I}}$ by setting them to $x_{\mathcal{I}}$ is given by

$$P_\theta(X_{-\mathcal{I}} \mid \mathrm{do}(X_{\mathcal{I}} = x_{\mathcal{I}})) = \mathbb{1}(X_{\mathcal{I}} = x_{\mathcal{I}}) \prod_{i \notin \mathcal{I}} P(X_i \mid \mathrm{Pa}_G(X_i); \theta_i). \tag{1.5.1}$$

We use the notation "$\mathrm{do}(X_{\mathcal{I}} = x_{\mathcal{I}})$" to distinguish it from standard conditioning, and to make the active nature of an intervention more explicit (as in *doing* an action). From this equation, it is clear what is the effect of an intervention on the distribution: it sets the values of the variables $X_{\mathcal{I}}$, disregarding the values of their parents in G. From a graphical point of view, this can be seen as encoding this distribution with a *mutilated DAG*, where the incoming edges to $X_{\mathcal{I}}$ have been removed; see Figure 1.3. While we saw that a Bayesian network encodes a joint distribution over $X_1, \ldots, X_d$, a



(a) $P(A, B, D, E, L, S, T, X)$



(b) $P(A, B, D, L, S, T, X \mid \mathrm{do}(E = e))$

Figure 1.3. The effect of an intervention on the causal graph. (a) Example of a causal graph representing a family of distributions, including the joint (observational) distribution $P(A, B, D, E, L, S, T, X)$. (b) The mutilated graph corresponding to intervening on the random variable E (and setting it to the value e); the incoming edges of E have been removed.

causal model effectively represents a whole *family* of distributions, one for each possible intervention on some random variables; this notably includes the "observational" distribution where no variable has been intervened on, matching the joint distribution represented in a Bayesian network. Data collected by intervening on the system will be called *interventional data*, as opposed to *observational data* (*e.g.*, the datasets of observations $\mathcal{D}$ we considered so far in previous sections).

Going back to the example of the weather and barometer, and using our new notations, we concluded that for some value of the pressure y, $P_\theta(X \mid \mathrm{do}(Y = y)) = P_\theta(X)$; intervening on the barometer has no effect on the weather. This is to be contrasted with the conditional distribution $P_\theta(X \mid Y = y) = P_\theta(X, Y = y)/P_\theta(Y = y)$, which only encodes statistical relationships; if y is large, then there is a higher probability of having good weather, and is generally distinct from $P_\theta(X)$.

1.5.2. Causal identifiability

In Section 1.4.1, we focused on the identifiability of the structure of a Bayesian network (or its Markov equivalence class), in the context of structure learning. But identifiability comes in widely different forms, especially when it comes to causality. For example in the context of *causal inference*, identifiability could also mean the ability to express an interventional query involving "do" operators with quantities that don't (*i.e.*, conditional distributions), which can then be estimated from data (Bareinboim et al., 2022). More recently, there has also been a growing interest for identifiability in the context of *causal representation learning* (Schölkopf et al., 2021; Lachapelle et al., 2023).

Going back to the question of structure identifiability, which is more aligned with the setting studied in this thesis, many of the considerations we had for Bayesian networks transfer directly to causal models as well. In particular, most of the tools for *causal structure learning* (also known as *causal discovery*) are derived from those we presented earlier. However, the presence of interventions may help identification of the causal structure (more precisely than at the level of a Markov equivalence class; Hauser and Bühlmann, 2012). This is notably useful when the dataset we use for structure learning also includes *interventional data*.

Part I

Generative Flow Networks

Chapter 2

Probabilistic Inference over Structured Objects

This chapter contains material from the following papers:

- *Moksh Jain,* ***Tristan Deleu****, Jason Hartford, Cheng-Hao Liu, Alex Hernandez-Garcia, Yoshua Bengio (2023).* GFlowNets for AI-Driven Scientific Discovery*. Digital Discovery, Royal Society of Chemistry.*
- ***Tristan Deleu****, Padideh Nouri, Nikolay Malkin, Doina Precup, Yoshua Bengio (2024).* Discrete Probabilistic Inference as Control in Multi-path Environments*. Conference on Uncertainty in Artificial Intelligence (UAI).*

In this thesis, we will put an emphasis on sampling from distributions defined over very structured objects, with applications to structure learning from a Bayesian perspective in the second part. We will see via multiple examples in this chapter how we can treat sampling from such distributions as a *sequential decision making* problem, which can eventually be solved as a control problem, albeit in limited cases. The objective of the first part of this thesis will then be to extend these ideas so that they can be applied more broadly.

2.1. Energy-based models

Throughout the first part of this thesis, we are interested in the broad question of sampling from an *energy-based model* (EBM; LeCun et al., 2006) over a *discrete* and *structured* sample space $\mathcal{X}$. The "structured" aspect of $\mathcal{X}$ will be further expanded in Section 2.1.3. Given an energy function $\mathcal{E}(x)$, which we assume to be known and fixed (*i.e.*, this is an oracle that can be queried for any $x \in \mathcal{X}$), our objective is to sample from the following *Gibbs distribution*:

$$P^{\star}(x) = \frac{\exp(-\mathcal{E}(x))}{Z}. \tag{2.1.1}$$

This energy function encodes some notion of *preference* for certain objects, which can come in different forms (*e.g.*, the compatibility between an image and a label, or between a sentence in

English and its candidate translation in French). It is often convenient to use an energy since it can be an arbitrary function that does not necessarily need to be normalized. All the complexity related to the normalization is deferred to the *partition function* $Z = \sum_{x' \in \mathcal{X}} \exp(-\mathcal{E}(x'))$, which is often intractable when the space $\mathcal{X}$ is combinatorially large, as is the case for the majority of applications we consider here.

2.1.1. Why sampling from an EBM?

Although the problem of sampling from (2.1.1) may seem somewhat restricted at first glance, it is in fact a fundamental component for inference and learning in energy-based models.

Sampling-based inference. Probabilistic inference as a general principle can be cast as computing quantities of the form $\mathbb{E}_{P^\star}[f(x)]$, with an appropriately chosen function f, and where the expectation is taken wrt. the Gibbs distribution. For example, if we want to find the probability $P^\star(x \in A)$ of an object having some property A, then we would use $f(x) = \mathbb{1}(x \in A)$ in our general form. Even if in certain specific cases exact inference may be possible (Wainwright and Jordan, 2008), typically these quantities can only be approximated since the expectation requires summing over all the elements of the (combinatorially large) space $\mathcal{X}$. If we have a process to obtain samples from (2.1.1), then we can obtain a *Monte Carlo* estimate of this expectation: if $\{x^{(1)}, \dots, x^{(N)}\}$ are iid. samples from $P^\star$, then

$$\mathbb{E}_{P^\star}[f(x)] \approx \frac{1}{N} \sum_{n=1}^{N} f(x^{(n)}). \tag{2.1.2}$$

This estimate of the query of interest is unbiased, and consistent by the law of large numbers. This is an alternative to using variational inference as described in Section 1.2.

Learning the energy function. In this paragraph only, we will assume that the energy function $\mathcal{E}_\theta(x)$ is parametrized by some unknown θ, that we would like to learn based on a dataset of observations $\mathcal{D} = \{x^{(1)}, \dots, x^{(N)}\}$. Learning the parameters θ can be done by maximizing the (average) log-likelihood of the data under the (now parametric) Gibbs distribution $P^\star_\theta(x)$

$$\mathcal{L}(\theta) \triangleq \mathbb{E}_{P_\mathcal{D}}[\log P^\star_\theta(x)] = -\frac{1}{N} \sum_{n=1}^{N} \mathcal{E}_\theta(x^{(n)}) - \log Z_\theta, \tag{2.1.3}$$

where $P_\mathcal{D}$ is the empirical distribution over the dataset $\mathcal{D}$. This can be maximized using gradient-based methods, as long as one can easily compute the gradient of $\mathcal{L}(\theta)$ wrt. the parameters θ. It can be shown (Hinton, 2002) that this gradient has a particularly simple form

$$\nabla_\theta \mathcal{L}(\theta) = -\mathbb{E}_{P_\mathcal{D}}[\nabla_\theta \mathcal{E}_\theta(x)] + \mathbb{E}_{P^\star_\theta}[\nabla_\theta \mathcal{E}_\theta(x)]. \tag{2.1.4}$$

While the first term can be easily computed by averaging over $\mathcal{D}$, provided one can compute $\nabla_\theta \mathcal{E}_\theta(x)$, the second term is an expectation under the Gibbs distribution itself, and we saw in (2.1.2) that we can obtain a Monte Carlo estimate of it if we had a way to sample from $P^\star_\theta$.

But beyond the sole interest in statistics, sampling plays a crucial role in scientific discovery in general, for example in the generation of diverse proteins with specific structures and properties (Watson et al., 2023; Huguet et al., 2024). Aligned with scientific discovery, sampling also plays an

Algorithm 1 Metropolis-Hastings algorithm

Inputs: An initial state $x^{(0)} \in \mathcal{X}$, the proposal distribution $Q(x' \mid x)$.

for $t \geq 0$ **do**

  Sample a proposed move: $x' \sim Q(x' \mid x^{(t)})$

  Compute the the acceptance probability $A(x^{(t)}, x')$ (2.1.5)

  Sample $u \sim U(0, 1)$

  Update the state:

$$x^{(t+1)} = \begin{cases} x' & \text{if } u \leq A(x^{(t)}, x') \quad \text{(accept)} \\ x^{(t)} & \text{otherwise} \qquad\quad \text{(reject)} \end{cases}$$

important role in *active learning*, where the most informative and promising elements are sampled for later evaluation. Finally, we cannot talk about sampling without mentioning the massive impact that *generative AI* has nowadays (Brown et al., 2020; Betker et al., 2023), with its seemingly endless potential but also its societal risks (OECD, 2024).

2.1.2. Sampling from an EBM with MCMC

One of the most versatile tool for sampling from an EBM (or distributions known up to a normalization constant at large) is a technique called *Markov chain Monte Carlo* (MCMC; Gelfand and Smith, 1990). The idea is to construct a Markov chain on the sample space $\mathcal{X}$ whose stationary distribution is the Gibbs distribution in (2.1.1), meaning that after running the chain for long enough the samples will eventually be distributed approximately as $P^\star$.

A popular strategy to build such a Markov chain is the *Metropolis-Hastings* algorithm (Metropolis et al., 1953; Hastings, 1970), where moves from the current state $x \in \mathcal{X}$ to a new state $x' \in \mathcal{X}$ are proposed by a known (and easy to sample from) distribution $Q(x' \mid x)$. This *proposal distribution* is typically a distribution over local "mutations" of x, *e.g.*, adding or removing a single edge in a graph; see also Section 7.1.1 for an example where $P^\star$ is a distribution over graphs. To ensure that the stationary distribution of this chain is indeed $P^\star$, the new state x' is only accepted ($x^{(t+1)} = x'$) randomly with probability $\min(1, A(x^{(t)}, x'))$, where

$$A(x^{(t)}, x') \triangleq \frac{P^\star(x')Q(x^{(t)} \mid x')}{P^\star(x^{(t)})Q(x' \mid x^{(t)})} = \frac{Q(x^{(t)} \mid x')}{Q(x' \mid x^{(t)})} \exp(\mathcal{E}(x^{(t)}) - \mathcal{E}(x')), \tag{2.1.5}$$

and otherwise, the state remains unchanged ($x^{(t+1)} = x^{(t)}$). We can observe that the acceptance probability A only depends on the difference in energies between $x^{(t)}$ and x', and not on the intractable partition function Z. For large enough $t \gg 1$, $x^{(t)}$ sampled using this process is approximately distributed as $P^\star$. The pseudo-code for the Metropolis-Hastings algorithm is available in Algorithm 1.

It is important to note that the quality of the samples obtained with MCMC highly depends on how close the Markov chain is to stationarity. The time it takes to reach this state, called the *mixing time*, depends on both the target distribution (2.1.1) (and by extension, the energy function $\mathcal{E}(x)$) and on the proposal distribution. For example, MCMC is known to converge slowly when the target distribution has multiple modes separated by large regions of low probability and $Q(x' \mid x)$ only

proposes local moves around x (Roberts and Tweedie, 2008). To maximize the chances of getting approximate samples of $P^\star$, it is common practice to discard early samples for a "burn-in" period. Although there is a vast literature on analysing and improving the convergence of MCMC methods (Swendsen and Wang, 1986; Robert et al., 2018), an exhaustive treatment is outside the scope of this thesis. Besides mixing, MCMC also suffers from *autocorrelation*: due to its sequential nature, samples may be highly correlated, making the estimation of quantities in (2.1.2) less efficient.

It cannot be understated how influential this algorithm has been (and MCMC in general). It has been considered one of the top 10 algorithms of the 20th century by the IEEE magazine *Computing in Science & Engineering* (Beichl and Sullivan, 2000), and has been the cornerstone of major scientific discoveries, including in high-energy physics and lattice field theory (Lautrup, 1985), the estimation of the effects of monetary policies in macroeconomy (which won the Nobel prize in economics in 2005; Sims, 2012), or the modeling of complex chemical systems at different scales (Minary and Levitt, 2010; which once again won the Nobel prize in chemistry in 2013; Levitt, 2014).

Using gradient information. In the standard version of the Metropolis-Hastings algorithm presented above, the proposal distribution $Q(x' \mid x)$ is not aware of the energy "surface" around the current state x (the difference in energies only appearing in (2.1.5)), meaning that the moves are non-adaptive. On the contrary, we could imagine proposing larger moves when $\mathcal{E}(x)$ has larger variations. Suppose in this paragraph only that $\mathcal{X} \subseteq \mathbb{R}^d$ is a continuous sample space, and that the energy function is differentiable; this is a setting which has been, by and large, the most studied in the EBM literature. We could use the *Langevin dynamics* in order to propose new moves, with

$$Q(x' \mid x) \propto \exp\left(-\frac{1}{4\beta}\|x' - x + \beta\nabla_x \mathcal{E}(x)\|^2\right), \tag{2.1.6}$$

where $\beta > 0$ is a fixed step-size. Using this proposal distribution in Algorithm 1 is called the *Metropolis-adjusted Langevin algorithm* (MALA; Roberts and Tweedie, 1996). Langevin Monte Carlo has recently gained popularity thanks to its applications to score-based models (Song and Ermon, 2019), which are closely related to EBMs. However, while there has been some works adapting these ideas to discrete spaces (Grathwohl et al., 2021; Zhang et al., 2022c), in general these methods cannot be applied to our setting where $\mathcal{X}$ is discrete and structured since they rely on the score function $\nabla_x \mathcal{E}(x)$, which is undefined in our case a priori.

2.1.3. Compositional objects

We focus our attention to cases where the sample space $\mathcal{X}$ of the EBM is discrete and structured. The structure of this space will often come from some *compositional* aspect of the objects to be generated, meaning that the elements $x \in \mathcal{X}$ can be decomposed into pieces, and those pieces can then be reassembled to create new objects. We provide multiple examples of compositional objects and the pieces they are made of in Figure 2.1. Of major interest for us in this thesis, we can for example view a directed acyclic graph (DAG) encoding the structure of a Bayesian network as a collection of edges connecting a fixed set of nodes. We keep this notion of compositionality rather vague on purpose to avoid being restrictive, and we may also consider somewhat loose notions of

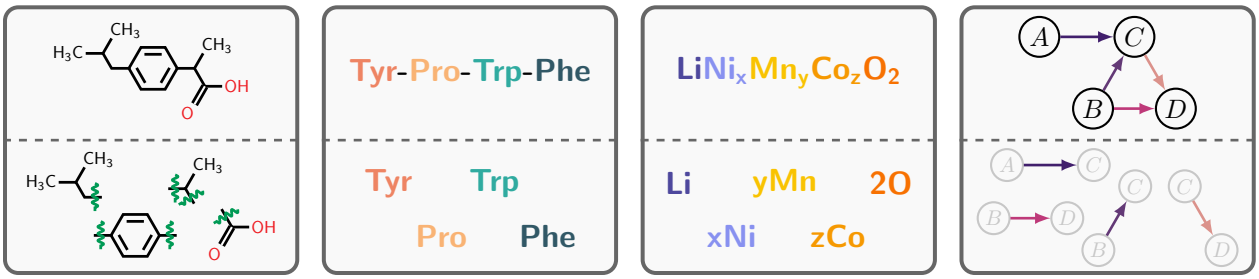


Figure 2.1. Examples of compositional objects. An organic molecule is a collection of molecular fragments assembled together. A protein sequence is a sequence of amino acids. An inorganic material is composed multiple elements (often organized as a crystal; Mila AI4Science et al., 2023). A directed acyclic graph is a collection of edges, combined together so that they don't form any cycle.

composition. For example in the context of natural language, a sentence may be viewed as being "compositional" in the sense that it is a sequence of words.

If we were to use MCMC methods in order to sample from a distribution over compositional objects, then the local "mutations" necessary to move in the sample space may correspond to replacing one piece with another, or adding / removing a piece from the object, *provided that these moves lead to another valid object* in $\mathcal{X}$. This is quite restrictive, and may lead to significant complexity to validate if a move is legal or not. For example, removing a fragment from a molecule may or may not be physically plausible. Applying MCMC in the context of DAGs will be detailed in Section 7.1.1.

2.2. Marginal sampling as sequential decision making

Instead of sampling an object directly from $P^\star$, possibly by moving in $\mathcal{X}$ with MCMC, we could leverage the compositional structure of each object $x \in \mathcal{X}$ and construct it *from scratch* by adding pieces together sequentially. In this section, we will show with multiple examples how we can sample from a complex distribution over compositional objects through a sequence of (simpler) decisions. We will also introduce notions of graph theory and probabilities which will be central in the rest of this thesis.

2.2.1. The Galton board

In his book on *Natural Inheritance* in 1889, Francis Galton proposed a mechanical device to illustrate what he called "the curve of frequency". This device, called the *Galton board* and illustrated in Figure 2.2a, is composed of a funnel at the top guiding beads into a series of pins disposed in a quincunx pattern. As the beads fall down due to gravity, they will bounce off each pin either to the left or to the right, going down each row to finally end up in one of the bins at the bottom of the apparatus. Galton (1889) observed that as the numbers of beads and bins increase, the shape outlined by the beads at the bottom of the board resembles a "*Normal curve of frequency*"; this is an illustration of what we now know as the *central limit theorem*.

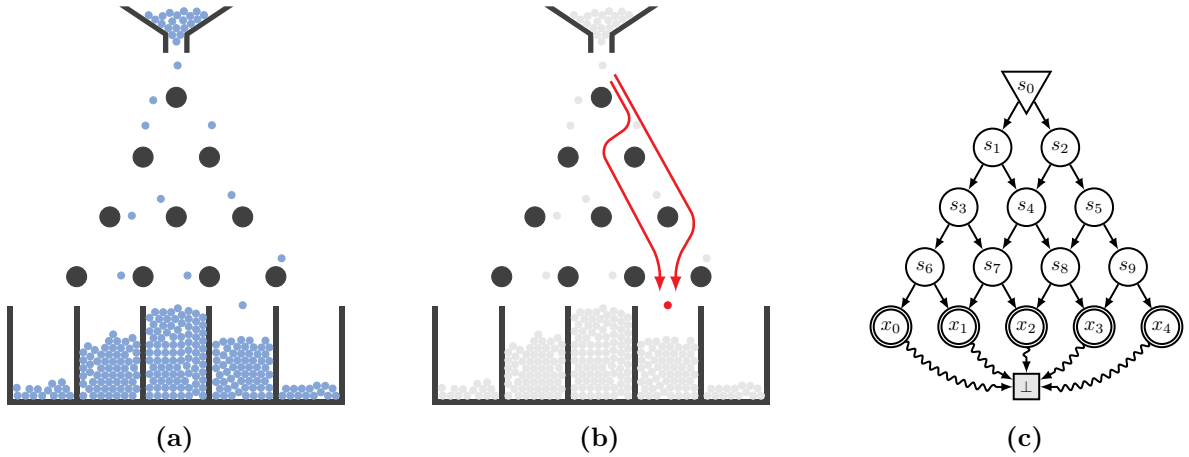


Figure 2.2. The Galton board. (a) Visualization of the Galton board, where blue beads fall down the apparatus into one of the five bins. (b) A bead falling into a specific bin may take multiple paths, bouncing off a different sequence of pins. (c) Representation of the Galton board as a pointed DAG $\mathcal{G}$. The intermediate states s_i represent the different pins organized in quicunx, and the terminating states x_k represent the bins.

Beyond this asymptotic curiosity, and the unsettling conclusions Galton drew from it, this model is simple enough that we can analyze its behavior with a fixed number of $n+1$ bins denoted by $\{0, \dots, n\}$. Assuming that a certain bead has probability p of going left and $1-p$ of going right each time it bounces off a pin (in reality, $p = 0.5$), it is well known that the bins in which the bead will fall is a random variable B that is distributed as a *binomial distribution*, where

$$P(B = k) = \binom{n}{k} p^k (1-p)^{n-k}. \tag{2.2.1}$$

While the binomial distribution could be written in terms of an energy function and its sample space $\mathcal{X} = \{0, \dots, n\}$ is discrete, this example deviates from our setting highlighted in Section 2.1.3 in that $\mathcal{X}$ has no apparent structure. Nevertheless, this serves as an intuitive illustration of a general principle that we will use extensively in the rest of this thesis:

> *We have a process to get samples from a more complex distribution as a sequence of multiple simple decisions.*

In our example, the distribution we sample from is arguably complex, since a bead falls in a certain bin with distribution Binomial(n, p), the sequence of decisions corresponding to going left at each pin with probability p (or Bernoulli(p)).

A second general principle is that we are interested in the *marginal distribution* over which bins the beads fall into, rather than the distribution over the exact paths the beads may take (*i.e.*, how they get to a certain bin).

> *We are only interested in the outcomes of the sequential process, not in the precise sequences of decisions themselves.*

As an example when $p = 0.5$, beads tend to fall in the middle bins with higher probability since there are many different paths they could follow to reach those bins (see also Figure 2.2b); this

is reflected by the binomial coefficient in (2.2.1) counting the number of paths to bin k. Playing around with the distributions of each individual decision leads to various marginals at the end of this process; our objective will be eventually to carefully tune each of these distributions so that this marginal matches a complex distribution of interest.

2.2.2. Elements of graph theory

To formalize the example of the Galton board, and in preparation for more complex scenarios, we need to introduce a few notions of graph theory. We represent each decision point (*e.g.*, each pin) as a state s in a state space $\mathcal{S}$, connected together as a *pointed directed acyclic graph* to represent the outcomes of each decision (see also Figure 2.2c).

Definition 2.2.1 (Pointed DAG)**.** Let $\overline{\mathcal{S}} = \mathcal{S} \cup \{\bot\}$ be a state space $\mathcal{S}$ augmented with a terminal state $\bot \notin \mathcal{S}$, and let $\mathcal{A} \subseteq \mathcal{S} \times \overline{\mathcal{S}}$ be a set of edges connecting these states. A (connected) directed acyclic graph (DAG) $\mathcal{G} = (\overline{\mathcal{S}}, \mathcal{A})$ is called a *pointed DAG* if there exists a unique *initial state* $s_0 \in \mathcal{S}$ such that s_0 is a root of the DAG (*i.e.*, it has no parent), and $\bot$ is the unique state with no child.

Unlike most works on generative flow networks (Bengio et al., 2023), where the terminal state is denoted by "s_f", throughout this thesis we will use "$\bot$" to represent the terminal state, as in Lahlou et al. (2023). We use a symbol distinct from the notation "s" (used for the states of $\mathcal{S}$) to emphasize that the terminal state is *not* an element of the state space (albeit an element of the *augmented* space $\overline{\mathcal{S}}$). We will always assume that all the states of $\mathcal{G}$ are accessible from s_0, and that the terminal state $\bot$ is also accessible from all the states of $\mathcal{G}$. We denote by $\mathrm{Pa}_{\mathcal{G}}(s)$ and $\mathrm{Ch}_{\mathcal{G}}(s)$ respectively the parents and the children of a state $s \in \overline{\mathcal{S}}$ in $\mathcal{G}$.

The bins of the Galton board are also represented as states in this graph (*i.e.*, $\mathcal{X} \subseteq \mathcal{S}$); in this example, those states only have a single decision leading to the terminal state, signaling the end of the sequential process. The states where termination is possible are called *terminating states*.

Definition 2.2.2 (Terminating states)**.** Let $\mathcal{G} = (\overline{\mathcal{S}}, \mathcal{A})$ be a pointed DAG. The set of *terminating states* $\mathcal{X} \subseteq \mathcal{S}$ of $\mathcal{G}$ is defined as the parents of the terminal state in $\mathcal{G}$: $\mathcal{X} \triangleq \mathrm{Pa}_{\mathcal{G}}(\bot)$.

Based on our example in the previous section, we can see why the choice of $\mathcal{X}$, the sample space of the distribution of interest, to denote the set of terminating states is not coincidental. While in that example all the terminating states have $\bot$ as their unique child (*i.e.*, we are forced to terminate once the bead reaches one of the bins), it is important to note that in general reaching a terminating state does not necessarily force termination, only taking the transition $x \to \bot$ to the terminal state does. In other words, it is completely possible that a pointed DAG contains edges from a terminating state to some non-terminal $s \in \mathcal{S}$; see Figure 3.1 further for an example. The sequences of decisions taken to reach a certain terminating state (*e.g.*, the pins a certain bead bounced off of) are encoded through *trajectories* in $\mathcal{G}$.

Definition 2.2.3 (Complete trajectories)**.** Let $\mathcal{G} = (\overline{\mathcal{S}}, \mathcal{A})$ be a pointed DAG, with s_0 its initial state. The trajectory $\tau = (s_0, s_1, \dots, s_T, \bot)$ is called a *complete trajectory* if for all $t \geq 0$, $s_t \to s_{t+1} \in \mathcal{A}$ (with the convention $s_{T+1} = \bot$). The set of all the complete trajectories over $\mathcal{G}$ is denoted by $\mathcal{T}$.

It is clear that for a complete trajectory $\tau = (s_0, s_1, \dots, s_T, \bot)$, $s_T \in \mathcal{X}$ is a terminating state. In some cases, it will also be convenient to talk about *partial trajectories*, which are defined similarly except that the start and end points of those trajectories are not necessarily the initial state s_0 or the terminal state $\bot$ respectively. We will use "$s_m \rightsquigarrow s_n$" to denote the set of partial trajectories from $s_m \in \mathcal{S}$ to $s_n \in \overline{\mathcal{S}}$ (s_n may be the terminal state); in particular, it is clear that $\mathcal{T} \equiv s_0 \rightsquigarrow \bot$. If τ and τ' are two partial trajectories such that the end point of τ matches the start point of τ', we will denote by $\tau \oplus \tau'$ the concatenation of both trajectories. We will also use the abuse of notation "$s \to s' \in \mathcal{G}$" when the context is clear to denote $s \to s' \in \mathcal{A}$, where $\mathcal{A}$ is the set of edges of $\mathcal{G}$.

Notations. We will often visualize a pointed DAG $\mathcal{G}$ as a graphical structure like the one shown in Figure 2.2c. We will use standard circles ○ to denote the states in $\mathcal{S}$, and double circles ◎ to highlight the terminating states in $\mathcal{X}$. We identify the initial state s_0 with a triangular node ▷, and the terminal state as ⊥. We use ⇝ to denote the terminating edges leading to $\bot$. Note that when the context is clear (*e.g.*, in Figure 2.3), we may duplicate the node ⊥ for visual clarity, even though it must be understood as being *one single state* $\bot$. We will give a summary of these notations in the annotated Figure 3.1b. In the second part of this thesis, we will relax these notations (*e.g.*, the terminal state will be implicit, as in Figure 7.2).

2.2.3. Forward transition probabilities

In addition to the structure of the state space encoding the outcomes of each decision made sequentially, we also need to specify *how* these decisions are made. For example in the Galton board, each bead had probability p of falling to the left if it bounced off a pin. We can formalize this with a probability distribution to transition from any state to the next states (*e.g.*, transitioning with a Bernoulli(p)).

Definition 2.2.4 (Consistent forward transition probabilities)**.** Let $\mathcal{G} = (\overline{\mathcal{S}}, \mathcal{A})$ be a pointed DAG. A function $P_F : \mathcal{S} \to \Delta(\mathrm{Ch}_{\mathcal{G}})$ is called a *forward transition probability* distribution consistent with $\mathcal{G}$ if for any $s \in \mathcal{S}$, $P_F(\cdot \mid s)$ is a properly defined distribution over $\mathrm{Ch}_{\mathcal{G}}(s)$, *i.e.*, for all $s' \in \mathrm{Ch}_{\mathcal{G}}(s)$, $P_F(s' \mid s) \geq 0$ and

$$\sum_{s' \in \mathrm{Ch}_{\mathcal{G}}(s)} P_F(s' \mid s) = 1. \tag{2.2.2}$$

In the definition above, for some state $s \in \mathcal{S}$, we use the notation "$\Delta(\mathrm{Ch}_{\mathcal{G}})$" to denote the space of probability distributions over the children of s in $\mathcal{G}$. In the context of reinforcement learning, a forward transition probability is also called a *policy* (Puterman, 1994), provided the underlying environment is deterministic (*i.e.*, there is a one-to-one mapping between actions and next states).

Although we will use "policies" in Section 2.3, the term "forward transition probability" will be prevalent in Chapters 3 & 4. We can also naturally extend the definition of P_F for some partial trajectory $\tau = (s_m, s_{m+1}, \dots, s_n)$ as

$$P_F(\tau) = \prod_{t=m}^{n-1} P_F(s_{t+1} \mid s_t). \tag{2.2.3}$$

Consistent forward transition probabilities have the interesting property that they also induce a distribution over partial trajectories starting at any state s and ending at $\perp$.

Lemma 2.2.5 (Distribution over suffixes)**.** *Let $\mathcal{G} = (\overline{\mathcal{S}}, \mathcal{A})$ be a pointed DAG, and let $P_F : \mathcal{S} \to \Delta(\mathrm{Ch}_{\mathcal{G}})$ be a forward transition probability consistent with $\mathcal{G}$. For any state $s \in \mathcal{S}$, P_F induces a distribution over partial trajectories τ starting at s and ending at the terminal state $\perp$, i.e., $P_F(\tau) \geq 0$ and*

$$\sum_{\tau : s \rightsquigarrow \perp} P_F(\tau) = 1. \tag{2.2.4}$$

Proof. Since P_F is a forward transition probability, we clearly have $P_F(\tau) \geq 0$. We can prove (2.2.4) by strong induction on the maximum length of the partial trajectories. For any state $s \in \mathcal{S}$, let d_s denote the maximum length of a partial trajectory from s to the terminal state $\perp$.

Base case: Since $\mathcal{G}$ is a pointed DAG, there exists at least a state $s \in \mathcal{S}$ such that $d_s = 1$. In that case, $\perp$ is the only child of s (otherwise there would be a longer trajectory from s to $\perp$, making $d_s > 1$), and there exists a single partial trajectory of the form $s \rightsquigarrow \perp$, which is the transition $s \to \perp$. Therefore

$$\sum_{\tau : s \rightsquigarrow \perp} P_F(\tau) = P_F(\perp \mid s) = 1. \tag{2.2.5}$$

Induction step: Suppose that (2.2.4) is valid for all states s' such that $d_{s'} \leq d$ for some $d > 0$. Since $\mathcal{G}$ is a pointed DAG, there exists a state $s \in \mathcal{S}$ such that $d_s = d + 1$. We can decompose the set of all partial trajectories from s into a disjoint union of partial trajectories going through each child $s' \in \mathrm{Ch}_{\mathcal{G}}(s)$ as

$$\{\tau : s \rightsquigarrow \perp\} = \bigcup_{s' \in \mathrm{Ch}_{\mathcal{G}}(s)} \{(s \to s') \oplus \tau' \mid \tau' : s' \rightsquigarrow \perp\}. \tag{2.2.6}$$

Since all the partial trajectories of the form $s' \rightsquigarrow \perp$ have a maximum depth $d_{s'} \leq d$, we can apply the induction step to them. Using the decomposition above, we get

$$\sum_{\tau : s \rightsquigarrow \perp} P_F(\tau) = \sum_{s' \in \mathrm{Ch}_{\mathcal{G}}(s)} P_F(s' \mid s) \sum_{\tau' : s' \rightsquigarrow \perp} P_F(\tau') = \sum_{s' \in \mathrm{Ch}_{\mathcal{G}}(s)} P_F(s' \mid s) = 1. \tag{2.2.7}$$

This proves that (2.2.4) is also valid for a state s with maximum trajectory length $d + 1$. □

A direct consequence of this lemma is that P_F induces in particular a probability distribution over complete trajectories (*i.e.*, starting at s_0). However as we noted in Section 2.2.1, we are interested in a distribution over terminating states (*i.e.*, a distribution over the bins in which the beads fall)

derived from this distribution over complete trajectories, and not $P_F(\tau)$ itself. This is called the *terminating state probability* distribution.

Definition 2.2.6 (Terminating state probability)**.** Let $\mathcal{G} = (\overline{\mathcal{S}}, \mathcal{A})$ be a pointed DAG, and $\mathcal{X} \subseteq \mathcal{S}$ be the set of terminating states in $\mathcal{G}$. Let $P_F : \mathcal{S} \to \Delta(\mathrm{Ch}_{\mathcal{G}})$ be an arbitrary forward transition probability consistent with $\mathcal{G}$. The *terminating state probability* distribution $P_F^\top$ (associated with P_F) is defined for all $x \in \mathcal{X}$ as

$$P_F^\top(x) \triangleq \sum_{\tau : x \to \perp \in \tau} P_F(\tau) = \sum_{\tau : s_0 \rightsquigarrow x} \prod_{t=0}^{T_\tau} P_F(s_{t+1} \mid s_t), \tag{2.2.8}$$

with the conventions $s_{T_\tau+1} = \perp$, and necessarily $s_{T_\tau} = x$ for all complete trajectories τ such that $x \to \perp \in \tau$.

Putting it in words, the terminating state probability distribution is the marginal distribution of $P_F(\tau)$ over trajectories terminating at x (*i.e.*, $x \to \perp \in \tau$). It is important to note that this is different from the marginal over trajectories simply going through x, since these are not necessarily guaranteed to directly take the transition $x \to \perp$. For example in Figure 3.1 further, although the trajectory $(s_0, s_2, x_4, x_6, \perp)$ passes through the terminating state x_4, it would not contribute to $P_F^\top(x_4)$. The following proposition guarantees that for any forward transition probability P_F, its corresponding terminating state probability is a properly defined distribution over the set of terminating states $\mathcal{X}$.

Proposition 2.2.7. *The terminating state probability distribution $P_F^\top$ is a well-defined probability distribution over the terminating states, in the sense that for all $x \in \mathcal{X}$, $P_F^\top(x) \geq 0$, and $\sum_{x \in \mathcal{X}} P_F^\top(x) = 1$.*

Proof. Since P_F is a forward transition probability, it is clear that $P_F^\top(x) \geq 0$ for any $x \in \mathcal{X}$. The normalization is a consequence of Lemma 2.2.5 applied to the initial state s_0:

$$\sum_{x \in \mathcal{X}} P_F^\top(x) = \sum_{x \in \mathcal{X}} \sum_{\tau : x \to \perp \in \tau} P_F(\tau) = \sum_{\tau : s_0 \rightsquigarrow \perp} P_F(\tau) = 1. \tag{2.2.9}$$

□

Interpreting $P_F^\top$ as a marginal distribution suggests a natural procedure for sampling from it, detailed in Algorithm 2: (1) sample a complete trajectory following the forward transition probabilities P_F, and (2) return only the terminating state reached at the end of the trajectory, right before transitioning to $\perp$. This is the same procedure as sampling from $P_F^\top = \mathrm{Binomial}(n, p)$ associated with $P_F(\cdot \mid s) = \mathrm{Bernoulli}(p)$ by letting a bead fall down the Galton board, and returning the bin in which it eventually fell into. Since $\mathcal{G}$ is acyclic, this procedure is guaranteed to terminate in time $O(|\mathcal{S}|)$ (worst case). However in many interesting cases, the "depth" of all the terminating states from the initial state s_0 will be significantly smaller than the total number of states, as in Figure 2.2,

Algorithm 2 Sampling from the terminating state probability distribution $P_F^\top$ – Definition 2.2.6

Inputs: A pointed DAG $\mathcal{G} = (\overline{\mathcal{S}}, \mathcal{A})$, a forward transition probability $P_F : \mathcal{S} \to \Delta(\mathrm{Ch}_{\mathcal{G}})$.
Outputs: A sample $x \sim P_F^\top$ of the terminating state probability distribution associated with P_F.

Initialize the state: s_0
repeat
    Sample the next state: $s_{t+1} \sim P_F(\cdot \mid s_t)$
until $s_{T+1} = \perp$
return $x \equiv s_T$

but even more so when the state space $\mathcal{S}$ has some compositional aspect which is reflected in the structure of $\mathcal{G}$.

2.2.4. Generation of small organic molecules

Even though the example of the Galton board in Section 2.2.1 is interesting to give intuitions about how a sequential process can induce a terminating state probability distribution, it is limited in that $\mathcal{X}$ is not a structured sample space. In this section, we consider a more realistic scenario where we would like to sample from a distribution over small organic molecules (*i.e.*, molecules containing C-H or C-C bonds). Molecules have a natural compositional structure as a combination of multiple fragments together, as illustrated in Figure 2.1. Since the number of possible molecules is extremely large (Fink et al., 2005), defining a distribution by assigning a probability for each individual molecule is practically impossible, due to the normalization constraint. It would be far simpler to just *ignore* the normalization, and assign a certain value for each molecule. In that case, the energy function $\mathcal{E}(x)$ in (2.1.1) may correspond to the binding affinity with a target protein for example, in order to generate molecules that are more likely to bind to this protein (*e.g.*, Bengio et al. (2021) used the binding affinity to the soluble epoxide hydrolase (sEH) protein, a well-studied protein playing a role in respiratory and heart disease (Imig and Hammock, 2009)).

Instead of using MCMC methods for sampling from this distribution (Seff et al., 2019; Xie et al., 2021), we could construct a molecule *from scratch* by adding one fragment at a time (You et al., 2018; Bengio et al., 2021), as shown in Figure 2.3. Starting from the empty state, we choose a fragment from some vocabulary to be added to a (possibly partial) molecule at each step, until we reach the terminal state. This example highlights a major difference with MCMC: the sequence of decisions leading to the generation of a full molecule may go through partial molecules that are not physically plausible, whereas MCMC methods would only allow moves from a full molecule to another full molecule (*i.e.*, moves proposed by $Q(x' \mid x)$ in Algorithm 1 are restricted to valid elements of $\mathcal{X}$). Using the terminology introduced in previous sections, the set of terminating states $\mathcal{X}$ corresponds to full molecules, and $\mathcal{S} \backslash \mathcal{X}$ is the set of partial molecules.

The advantage of having compositional objects, as noted in Section 2.1.3, becomes apparent here: having substructures shared between different molecules means that we have many states (decision points) along the complete trajectories to any full molecule, and therefore the complexity of the target Gibbs distribution can be broken down into a sequence of much simpler distributions ($P_F(\cdot \mid s)$). This also reduces significantly the number of steps required to sample a full molecule using Algorithm 2.

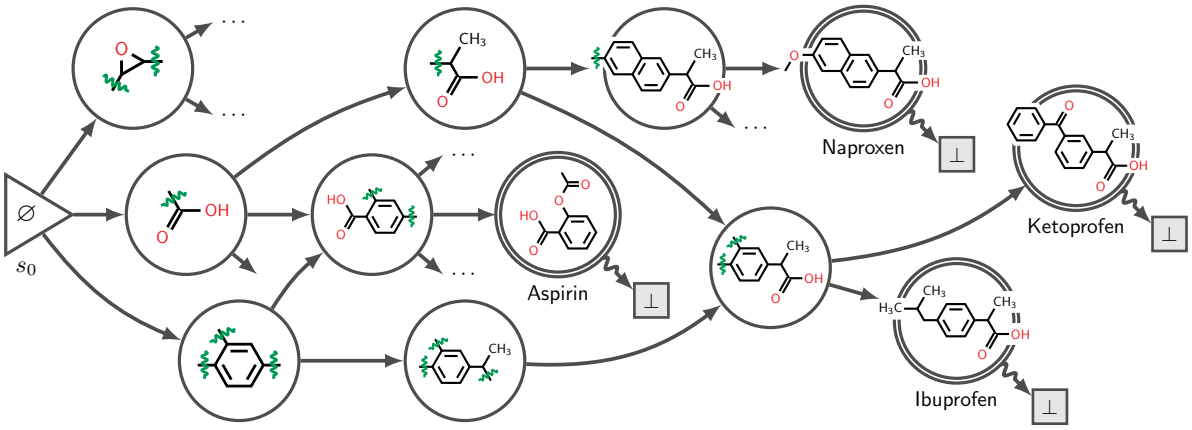


Figure 2.3. Pointed DAG for the generation of small organic molecules. At each step of generation, a new molecular fragment chosen from a fixed vocabulary of fragments is added at some end of the partial molecule. The process eventually leads to fully formed molecules (*e.g.*, aspirin), where termination is possible. Following a forward transition probability distribution P_F defined on this graph yields a terminating state probability distribution $P_F^\top$ over fully formed molecules.

However, there is a trade-off between the length of the complete trajectories and the complexity of the transition probability at each state: while we could imagine having a naive pointed DAG $\mathcal{G}$ where s_0 is directly connected to all the elements of $\mathcal{X}$ (*e.g.*, all the full molecules), this would make $P_F(\cdot \mid s_0)$ extremely complex (in fact, we would be back to finding $P^\star$ directly). We will see in Section 2.3 and subsequent chapters how to find a forward transition probability whose corresponding terminating state distribution $P_F^\top$ matches (2.1.1) for a given energy function $\mathcal{E}(x)$.

2.2.5. Autoregressive sampling from a factor graph

As a final example, this time anchored in the field of probabilistic modeling, we consider the problem of inference in a factor graph as introduced by Buesing et al. (2020). Suppose that we have d discrete random variables $(X_1, \dots, X_d)$, each X_i taking values in $\{1, \dots, K\}$, meaning that the sample space $\mathcal{X} = \{1, \dots, K\}^d$ is exponentially large. We assume that the joint distribution of these random variables is given by a *factor graph*, which can be expressed as a Gibbs distribution like in (2.1.1), with the energy function

$$\mathcal{E}(x_1, \dots, x_d) = \sum_{m=1}^{M} \psi_m(x_{[m]}). \tag{2.2.10}$$

Each factor ψ_m is a known function that only depends on a subset of random variables (the values of which are denoted by "$x_{[m]}$"). A factor graph can be represented as a bipartite graph whose nodes are the random variables $\{X_1, \dots, X_d\}$ on the one hand, and the factors $\{\psi_1, \dots, \psi_M\}$ on the other hand; see Figure 2.4 for an example of a factor graph with $d = 3$ variables and $M = 3$ factors connecting them.

Inference in factor graphs is a very well studied topic, whether it is exact (Dechter, 1999) or approximate (Murphy et al., 1999; Kschischang et al., 2001; De Sa et al., 2015). As an alternative, and similar to our previous examples, Buesing et al. (2020) treated sampling from this factor graph as a sequential decision making problem, where the value of each variable is assigned one at a time

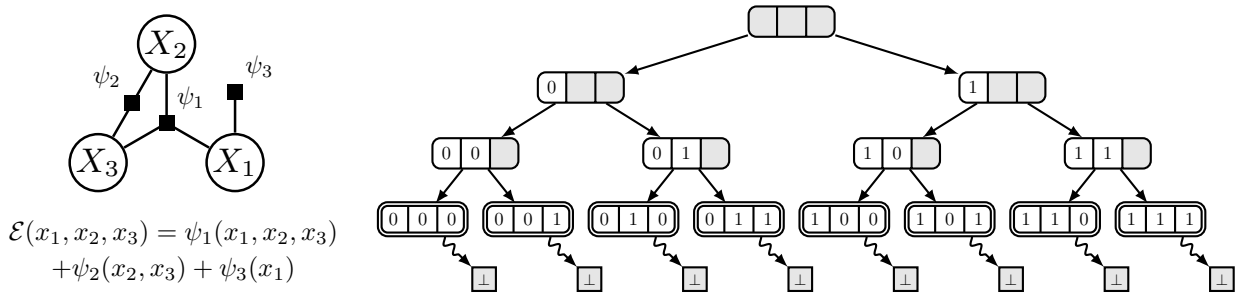


Figure 2.4. Autoregressive sampling from a discrete factor graph, viewed from the perspective of sequential generation in a pointed DAG. (Left) An example of a factor graph over 3 binary random variables X_1, X_2, and X_3, with 3 factors ψ_1, ψ_2, and ψ_3. The energy function associated with the distribution represented by this factor graph is the sum of its factors. (Right) Representation of the sequential generation of a full assignment of the variables (x_1, x_2, x_3) as a pointed DAG $\mathcal{G}$. $\perp$ is displayed as separate nodes for clarity only, and should be understood as a single state.

following a fixed order. Figure 2.4 shows the pointed DAG $\mathcal{G}$ for a factor graph with binary variables ($K = 2$), where we first assign the value of X_1, then X_2, and finally X_3. Therefore to obtain a full sample from $P^\star(x)$, this process requires a sequence of exactly d steps. Interestingly, and this time unlike our previous examples, $\mathcal{G}$ has a tree structure (with the exception of the terminal state $\perp$, whose parents are always all the terminating states in $\mathcal{X}$ by Definition 2.2.2), which is typically the case for autoregressive sequence generation tasks (Bachman and Precup, 2015; Weber et al., 2015; Angermueller et al., 2020; Jain et al., 2022; Feng et al., 2022; Rafailov et al., 2024).

2.3. Probabilistic inference as a control problem

Continuing with our example in Section 2.2.5, if the forward transition probability P_F was known, following this distribution would yield samples from the terminating state distribution $P_F^\top$. However, we don't want to sample from any distribution, but very specifically from the Gibbs distribution with the energy function in (2.2.10). How can we find P_F so that $P_F^\top$ matches this distribution of interest, for a given energy function $\mathcal{E}(x)$? In this section, we will show that we can treat this as a control problem within the framework of entropy-regularized reinforcement learning (Ziebart, 2010; Fox et al., 2016). We will use "policies" and denote them by "π" (instead of "forward transition probabilities" and "P_F") to be more aligned with the naming conventions in reinforcement learning.

2.3.1. Maximum entropy reinforcement learning

We recall some notions of (maximum entropy) reinforcement learning (Sutton and Barto, 2018), with notations and concepts adapted to our setting. We consider a finite-horizon Markov Decision Process (MDP) $\mathcal{M} = (\mathcal{G}, r)$, where $\mathcal{G} = (\overline{\mathcal{S}}, \mathcal{A})$ is a pointed DAG specifying the structure of the MDP, and r is a specific reward function defined below. $\mathcal{S}$ plays the role of the state space and $\mathcal{A}$ the action space of the MDP, both being discrete and finite. The MDP is deterministic, in the sense that we are guaranteed to transition to a certain state s' from a state $s \in \mathcal{S}$ after following an action $a = s \to s' \in \mathcal{A}$. Since $\mathcal{G}$ is a pointed DAG, all the (complete) trajectories in this MDP start

at the initial state s_0, and are guaranteed to eventually end at the terminal state $\perp$ (finite-horizon). We also assume that we are in an undiscounted regime, meaning that the discount factor $\gamma = 1$.

Since the MDP is deterministic, we can identify the action leading to a new state s' from s with the transition $s \to s' \in \mathcal{G}$ itself. As such, we will write all quantities involving state-action pairs (s, a) where $a \in \mathcal{A}$ (*e.g.*, the rewards, or the state-action values) directly in terms of (s, s') instead. The reward function $r(s, s')$ of this MDP is defined such that the sum of rewards along a complete trajectory (also called the *return*) only depends on the energy of the terminating state it reaches: for a trajectory $\tau = (s_0, s_1, \dots, s_T, \perp)$, we have

$$\sum_{t=0}^{T} r(s_t, s_{t+1}) = -\mathcal{E}(s_T), \tag{2.3.1}$$

with the convention $s_{T+1} = \perp$. In particular, this covers the case of a sparse reward that is received only at the end of the trajectory (*i.e.*, $r(s_T, \perp) = -\mathcal{E}(s_T)$, and zero everywhere else). In reinforcement learning, the objective is in general to find an *optimal policy* $\pi^\star_{\mathrm{RL}}(s' \mid s)$ that maximizes the expected return

$$\pi^\star_{\mathrm{RL}} = \arg\max_{\pi} \mathbb{E}_\tau \left[\sum_{t=0}^{T} r(s_t, s_{t+1}) \right], \tag{2.3.2}$$

where the expectation is taken wrt. the distribution $\pi(\tau)$ over complete trajectories induced by π (see Section 2.2.3). With our particular choice of reward function in (2.3.1), this maximization would correspond to finding a policy (that might be deterministic) that would lead to a state $x \in \mathcal{X}$ with the lowest energy here, or equivalently finding a mode of the Gibbs distribution $P^\star$. This differs from our initial goal of *sampling* from the whole distribution $P^\star$, and not only getting the objects with highest probability.

To account for possibly suboptimal objects, we need to regularize the objective in (2.3.2). In maximum entropy reinforcement learning (MaxEnt RL; Ziebart, 2010; Fox et al., 2016), our objective is instead to search for a *stochastic* policy that also maximizes the expected return, and the entropy $\mathcal{H}(\pi(\cdot \mid s)) = -\sum_{s' \in \mathrm{Ch}_{\mathcal{G}}(s)} \pi(s' \mid s) \log \pi(s' \mid s)$ of the policy π in state s, in order to encourage diversity of the actions taken:

$$\pi^\star_{\mathrm{MaxEnt}} = \arg\max_{\pi} \mathbb{E}_\tau \left[\sum_{t=0}^{T} r(s_t, s_{t+1}) + \mathcal{H}(\pi(\cdot \mid s_t)) \right], \tag{2.3.3}$$

where the expectation is again taken wrt. $\pi(\tau)$. In general, there may be a temperature parameter $\alpha > 0$ that controls the importance of the entropy regularization relative to the original objective, which we assume to be equal to 1 here for simplicity; we will come back to the impact of the temperature parameter in Chapter 5. This type of entropy regularization falls into the broader domain of *regularized MDPs* (Geist et al., 2019), and we will see another example of regularization based on the relative entropy in Section 5.3. MaxEnt RL has been proven to be particularly beneficial for improving exploration (Haarnoja et al., 2017) and for robust control under model misspecification (Eysenbach and Levine, 2022).

Haarnoja et al. (2017) showed that the policy maximizing the objective in (2.3.3) can be written explicitly as $\pi^\star_{\text{MaxEnt}}(s' \mid s) = \exp\left(Q^\star_{\text{soft}}(s, s') - V^\star_{\text{soft}}(s)\right)$, where $Q^\star_{\text{soft}}$ is the soft state-action value function and $V^\star_{\text{soft}}$ the soft state value function which satisfy the following *soft Bellman optimality equations* (adapted to our setting, *i.e.*, undiscounted and deterministic MDP):

$$Q^\star_{\text{soft}}(s, s') = r(s, s') + V^\star_{\text{soft}}(s') \tag{2.3.4}$$

$$V^\star_{\text{soft}}(s) = \log \sum_{s'' \in \text{Ch}_\mathcal{G}(s)} \exp\left(Q^\star_{\text{soft}}(s, s'')\right). \tag{2.3.5}$$

These equations differ from the standard Bellman optimality equations (Sutton and Barto, 2018) in that the the greedy operation appearing in the optimal policy $\pi^\star_{\text{RL}}$ is replaced by a softmax operation in $\pi^\star_{\text{MaxEnt}}$, and the "max" operation in the optimal state value function $V^\star$ is replaced by the log-sum-exp in (2.3.5).

2.3.2. Sampling terminating states from the soft MDP

From the literature on *control as inference* (Ziebart et al., 2008; Levine, 2018), it can be shown that the policy maximizing the MaxEnt RL objective in (2.3.3) induces a distribution over dynamically consistent trajectories that depends on the sum of rewards obtained along the trajectory.

Proposition 2.3.1. *Let $\mathcal{M}$ be a deterministic & undiscounted finite-horizon MDP. The policy $\pi^\star_{\text{MaxEnt}}$ maximizing (2.3.3) induces a distribution over complete trajectories $\tau = (s_0, s_1, \dots, s_T, \perp)$ such that*

$$\pi^\star_{\text{MaxEnt}}(\tau) = \prod_{t=0}^{T} \pi^\star_{\text{MaxEnt}}(s_{t+1} \mid s_t) \propto \exp\left(\sum_{t=0}^{T} r(s_t, s_{t+1})\right), \tag{2.3.6}$$

with the convention $s_{T+1} = \perp$.

Proof. Using the soft Bellman optimality equation (2.3.4) in $\pi^\star_{\text{MaxEnt}}$, we have

$$\pi^\star_{\text{MaxEnt}}(s' \mid s) = \exp\left(Q^\star_{\text{soft}}(s, s') - V^\star_{\text{soft}}(s)\right) = \exp\left(r(s, s') + V^\star_{\text{soft}}(s') - V^\star_{\text{soft}}(s)\right). \tag{2.3.7}$$

By telescoping the state value functions, we can conclude that

$$\pi^\star_{\text{MaxEnt}}(\tau) = \prod_{t=0}^{T} \pi^\star_{\text{MaxEnt}}(s_{t+1} \mid s_t) = \prod_{t=0}^{T} \exp\left(r(s_t, s_{t+1}) + V^\star_{\text{soft}}(s_{t+1}) - V^\star_{\text{soft}}(s_t)\right) \tag{2.3.8}$$

$$= \exp\left(\sum_{t=0}^{T} \left(r(s_t, s_{t+1}) + V^\star_{\text{soft}}(s_{t+1}) - V^\star_{\text{soft}}(s_t)\right)\right) \tag{2.3.9}$$

$$= \exp\left(\sum_{t=0}^{T} r(s_t, s_{t+1}) + V^\star_{\text{soft}}(\perp) - V^\star_{\text{soft}}(s_0)\right) \tag{2.3.10}$$

$$= \frac{1}{\exp\left(V^\star_{\text{soft}}(s_0)\right)} \exp\left(\sum_{t=0}^{T} r(s_t, s_{t+1})\right) \tag{2.3.11}$$

since $V^\star_{\text{soft}}(\perp) = 0$ (the terminal state has no child). $V^\star_{\text{soft}}(s_0)$ is a constant independent of τ. □

Algorithm 3 Soft Q-Learning algorithm

Inputs: A MDP $\mathcal{M} = (\mathcal{G}, r)$, a step size $\beta > 0$, a behavior policy $\pi_b(\cdot \mid s)$
Outputs: A stochastic policy $\pi^\star_{\text{MaxEnt}}$ maximizing (2.3.3)

1: Initialize $Q(s, s')$ for all $s \to s' \in \mathcal{G}$, such that $Q(x, \perp) = 0$ for all $x \in \mathcal{X}$.
2: **loop**
3: Start a trajectory at the initial state s_0
4: **repeat**
5: Sample the next state: $s_{t+1} \sim \text{Categorical}(\pi_b(\cdot \mid s_t))$
6: $Q(s_t, s_{t+1}) \leftarrow Q(s_t, s_{t+1}) + \beta\big[r(s_t, s_{t+1}) + \text{LogSumExp}_{s'} Q(s_{t+1}, s') - Q(s_t, s_{t+1})\big]$
7: **until** $s_{t+1} = \perp$
8: **return** $\pi^\star_{\text{MaxEnt}}(s' \mid s) \propto \exp\big(Q(s, s')\big)$

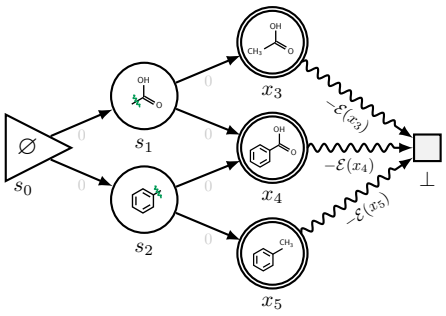
With our choice of reward function in (2.3.1), where the sum of rewards only depends on the energy of the terminating state, the distribution in Proposition 2.3.1 corresponds *almost* exactly to $P^\star$ in (2.1.1). However, the key difference between (2.3.6) and our goal of sampling from the Gibbs distribution is that $\pi^\star_{\text{MaxEnt}}(\tau)$ is a distribution over complete trajectories, while $P^\star$ is over $\mathcal{X}$ only. In other words, we are interested in the terminating state distribution $\pi^{\star\top}_{\text{MaxEnt}}(x)$, a distribution over the *outcomes* of the sequential process, and not a distribution over *how* we got there.

Fortunately when there is a unique complete trajectory $s_0 \rightsquigarrow x$ for any $x \in \mathcal{X}$, then the terminating state distribution matches exactly $\pi^\star_{\text{MaxEnt}}(\tau)$ (*i.e.*, there is only a single term in the sum appearing in Definition 2.2.6). For instance, this is the case in our factor graph inference example of Section 2.2.5 (see Figure 2.4), and this is true more generally whenever $\mathcal{G}$ has a tree structure. Therefore in those cases, if we know the policy $\pi^\star_{\text{MaxEnt}}$ maximizing (2.3.3), then we can sample from $P^\star$ by simply applying Algorithm 2 (*i.e.*, sampling a complete trajectory using this policy, and returning the state right before reaching $\perp$). Furthermore, unlike MCMC which produces autocorrelated samples, we can easily obtain independent samples from $P^\star$ by running this procedure multiple times.

Finding the optimal policy. To find $\pi^\star_{\text{MaxEnt}}$, we can then use any algorithm from the MaxEnt RL literature (Nachum et al., 2017; Haarnoja et al., 2018). Probably the simplest is the *Soft Q-Learning* algorithm (Haarnoja et al., 2017), which is the direct counterpart of the well-known Q-Learning algorithm in standard reinforcement learning (Watkins, 1989), whose objective is to find a state-action value function $Q(s, s')$ that satisfies the soft Bellman optimality equations; see Algorithm 3 for reference. Soft Q-Learning is an *off-policy* algorithm, meaning that we can use any arbitrary behavior policy π_b in order to transition to the next state s_{t+1} on line 5 (as opposed to being *on-policy*, where we would be forced to use the policy derived from the current $Q(s, s')$). In their application to inference in factor graphs described in Section 2.2.5, Buesing et al. (2020) leveraged the tree structure of $\mathcal{G}$ and used an algorithm based on *Monte Carlo tree search* (MCTS; Coulom, 2006) in order to find $\pi^\star_{\text{MaxEnt}}$.

2.3.3. Multi-path environments & biased sampling

We saw that when $\mathcal{G}$ has a tree structure, meaning that there is a unique complete trajectory going to any terminating state $x \in \mathcal{X}$, simply following $\pi^\star_{\text{MaxEnt}}$ yields samples from the Gibbs distribution if the reward is defined as (2.3.1). But when there are multiple ways of constructing a



x	τ	$\pi^\star_{\text{MaxEnt}}(\tau)$	$\pi^{\star\top}_{\text{MaxEnt}}(x)$
x_3	$s_0 \to s_1 \to x_3$	$\exp(-\mathcal{E}(x_3))/Z'$	$\exp(-\mathcal{E}(x_3))/Z'$
x_4	$s_0 \to s_1 \to x_4$	$\exp(-\mathcal{E}(x_4))/Z'$	$2\exp(-\mathcal{E}(x_4))/Z'$
	$s_0 \to s_2 \to x_4$	$\exp(-\mathcal{E}(x_4))/Z'$	
x_5	$s_0 \to s_2 \to x_5$	$\exp(-\mathcal{E}(x_5))/Z'$	$\exp(-\mathcal{E}(x_5))/Z'$

Figure 2.5. Illustration of the bias of the terminating state distribution associated with $\pi^\star_{\text{MaxEnt}}$ on a soft MDP with a DAG structure. The labels on each transition of the MDP corresponds to the reward function, satisfying (2.3.1) (*i.e.*, sparse reward setting). The terminating state distribution $\pi^{\star\top}_{\text{MaxEnt}}(x)$ is computed by marginalizing $\pi^\star_{\text{MaxEnt}}(\tau)$ over trajectories leading to x (*e.g.*, two trajectories $s_0 \to s_1 \to x_4$ and $s_0 \to s_2 \to x_4$ to x_4). $\pi^\star_{\text{MaxEnt}}(\tau)$ is computed based on Proposition 2.3.1. The terminating state distribution $\pi^{\star\top}_{\text{MaxEnt}}(x)$ should be contrasted with the Gibbs distribution $P^\star(x) \propto \exp(-\mathcal{E}(x))$ in (2.1.1). The normalization constant is $Z' = \exp(-\mathcal{E}(x_3)) + 2\exp(-\mathcal{E}(x_4)) + \exp(-\mathcal{E}(x_5))$. This is a simplified version of the molecule generation task described in Section 2.2.4.

terminating state, as in Sections 2.2.1 & 2.2.4, this does not hold any longer. It is important to recall that while the distribution over *how* the objects are generated (*i.e.*, the complete trajectories) is proportional to $\exp(-\mathcal{E}(x))$ by Proposition 2.3.1, we are interested in the (marginal) distribution over the *outcomes* (*i.e.*, the corresponding terminating state distribution). In fact, it is easy to show (based on Definition 2.2.6 and Proposition 2.3.1) that in general the terminating state distribution associated with $\pi^\star_{\text{MaxEnt}}$ is given by

$$\pi^{\star\top}_{\text{MaxEnt}}(x) = \sum_{\tau : x \to \perp \in \tau} \pi^\star_{\text{MaxEnt}}(\tau) \propto n(x) \exp\left(\sum_{t=0}^{T} r(s_t, s_{t+1})\right), \tag{2.3.12}$$

where $n(x) = |\{\tau \in \mathcal{T} \mid x \to \perp \in \tau\}|$ is the number of complete trajectories that terminate at x, and where $s_T = x$ and $s_{T+1} = \perp$ (Bengio et al., 2021). With the reward function $r(s, s')$ defined as (2.3.1), this means that $\pi^{\star\top}_{\text{MaxEnt}}(x) \propto n(x)\exp(-\mathcal{E}(x))$, which does *not* correspond to our target distribution (2.1.1). As an illustrative example, we consider a simplified version of the small molecule generation example of Section 2.2.4 in Figure 2.5, where the sample space only contains 3 molecules. If we follow the optimal policy $\pi^\star_{\text{MaxEnt}}$, the molecule x_4 will be artificially over-represented since it has 2 complete trajectories leading to it.

Our objective in the first part of this thesis is to build a framework that does not suffer from this bias anymore. Our goal will still be to find a policy whose terminating state distribution matches the Gibbs distribution of interest, but the approach will be *a priori* quite different from the control perspective we adopted in this section. This policy (or forward transition probability distribution P_F) will be derived from a *flow* in a flow network, as we will see in the following chapters, but we will eventually tie it back to MaxEnt RL in Chapter 5.

Chapter 3

Flow Networks

This chapter contains material from the following paper:

- *Yoshua Bengio*, Salem Lahlou*,* ***Tristan Deleu****, Edward Hu, Mo Tiwari, Emmanuel Bengio (2023).* GFlowNet Foundations. *Journal of Machine Learning Research (JMLR).*

In the previous chapter, we saw that when the objects have some compositional structure, we can naturally sample from the Gibbs distribution in (2.1.1) as a sequential decision making process through a pointed DAG $\mathcal{G}$ whose terminating states $\mathcal{X}$ is the sample space of $P^\star$. The main challenge was to find a forward transition probability P_F such that its corresponding terminating state distribution matched the target Gibbs distribution. We saw in Section 2.3 that naively applying (maximum entropy) reinforcement learning in order to find this P_F could lead to a distribution which is biased when $\mathcal{G}$ is a general DAG, with multiple ways of constructing the objects of interest. To avoid this bias, we will see in the next chapter how to find such a P_F in a general case for any structure $\mathcal{G}$. This solution will be based on a standard tool of graph theory called *flow networks*, of which we introduce the foundations in this chapter.

3.1. Flow networks & Markovian flows

What made the maximum entropy reinforcement learning approach fail in Section 2.3.3 was the possibility of having multiple paths leading to the same object in the pointed DAG $\mathcal{G}$, where the probabilities of each path were added together. This was effectively treating different paths going to the same object as being completely "independent", even though they could have some components in common (and they even share the same endpoint). To alleviate this issue, we need a structure that will take into account these dependencies between paths. This will come in the form of a *flow network*, which we will introduce in this section, along with its Markovian counterpart which will play a central role in practice when we will use them for generative modeling in Chapter 4.

3.1.1. Flow networks

In Section 2.2.2, we introduced the notion of pointed DAG $\mathcal{G}$ to encode the structure of a state space $\mathcal{S}$. We augment this DAG with a function $F^\star$ over complete trajectories of $\mathcal{G}$ called a *flow*.

Definition 3.1.1 (Trajectory flow)**.** Let $\mathcal{G}$ be a pointed DAG, and $\mathcal{T}$ be the set of complete trajectories over $\mathcal{G}$. A non-negative function over complete trajectories $F^\star : \mathcal{T} \to \mathbb{R}^+$ is called a *trajectory flow*. The set of all trajectory flows is denoted by $\overline{\mathcal{F}}(\mathcal{G}) \triangleq \mathcal{F}(\mathcal{T}, \mathbb{R}^+)$ (*i.e.*, the set of all non-negative functions over $\mathcal{T}$). The pair $(\mathcal{G}, F^\star)$ is called a *flow network*.

Although a trajectory flow $F^\star$ is defined as a function over complete trajectories, it can also naturally induce a measure over the measurable space of complete trajectories $(\mathcal{T}, \Sigma)$, where the σ-algebra $\Sigma = 2^{\mathcal{T}}$ is the power set of $\mathcal{T}$. We make an abuse of notation here and denote by $F^\star$ both the trajectory flow (a function), and its corresponding measure, where for every subset of complete trajectories $A \subseteq \mathcal{T}$, we have

$$F^\star(A) = \sum_{\tau \in A} F^\star(\tau). \tag{3.1.1}$$

In the special case where the subset A is a singleton trajectory $\{\tau\}$, we will simply write its measure as $F^\star(\tau)$ to identify it with the trajectory flow function. By another abuse of notation, we will use $F^\star$ to also denote flows going through specific states, transitions, or partial trajectories (see Section 3.1.4). For example, the *state flow* $F^\star(s)$ corresponds to the amount of flow going through the state $s \in \mathcal{S}$, and is defined by

$$F^\star(s) \triangleq F^\star(\{\tau \in \mathcal{T} \mid s \in \tau\}) = \sum_{\tau : s \in \tau} F^\star(\tau). \tag{3.1.2}$$

We will always assume that the trajectory flow $F^\star$ is non-identically zero, and that for any state $s \in \mathcal{S}$, there exists at least a trajectory going through s with positive flow, to guarantee that $F^\star(s) > 0$. Similarly, the *edge flow* $F^\star(s \to s')$ corresponds to the amount of flow going through the transition $s \to s' \in \mathcal{G}$:

$$F^\star(s \to s') \triangleq F^\star(\{\tau \in \mathcal{T} \mid s \to s' \in \tau\}) = \sum_{\tau : s \to s' \in \tau} F^\star(\tau). \tag{3.1.3}$$

These state and edge flows will play a crucial role throughout this section, and in particular the edge flows through terminating transitions of the form $F^\star(x \to \perp)$ for $x \in \mathcal{X}$, as we will see in Chapter 4. For any flow network, these flows are related via a "flow conservation" rule, which we will expand on in Section 3.2.

Proposition 3.1.2. *Given a flow network $(\mathcal{G}, F^\star)$, the state and edge flows satisfy*

$$\forall s \in \mathcal{S}, \qquad F^\star(s) = \sum_{s' \in \mathrm{Ch}_{\mathcal{G}}(s)} F^\star(s \to s') \tag{3.1.4}$$

$$\forall s' \neq s_0, \qquad F^\star(s') = \sum_{s \in \mathrm{Pa}_{\mathcal{G}}(s')} F^\star(s \to s') \tag{3.1.5}$$